

Міністерство освіти і науки України
Національний університет “Львівська політехніка”



Кафедра ЕОМ

**ДОСЛІДЖЕННЯ РЕЖИМІВ КЕРУВАННЯ КОМПОНЕНТАМИ МПС
З ДОПОМОГОЮ ПАРАЛЕЛЬНОГО ТА ПОСЛІДОВНОГО ІНТЕРФЕЙСІВ**

Методичні вказівки

до лабораторної роботи № 5 з курсу “Мікропроцесорні системи”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Затверджено
на засіданні
кафедри електронних обчислювальних машин
Протокол №1 від 29.08.2019р.

Львів – 2019

**Дослідження режимів керування компонентами МПС
з допомогою паралельного та послідовного інтерфейсів**

Методичні вказівки до лабораторної роботи № 5 дисципліни " Мікропроцесорні системи "
для студентів спеціальності 6.123.00.00 - "Комп'ютерна інженерія"

/ Укладач: Пуйда В.Я.,– Львів: Національний університет "Львівська політехніка", 2019, 44 с.

Укладач

Пуйда В. Я., к. т. н, доцент

Рецензент

Відповідальний за випуск:

Мельник А. О., професор, завідувач кафедри

Дослідження режимів керування компонентами МПС з допомогою паралельного та послідовного інтерфейсів

МЕТА РОБОТИ:

ознайомлення з технічними характеристиками та архітектурою мікроконтролера STM32F407, паралельними портами мікроконтролера STM32F407, інтерфейсом I2C, Flash FM24CL16, інтерфесом RS-485, керуванням світлодіодами та реле, основними режимами роботи інтегрованої системи mVision5 фірми KEIL, основними режимами роботи інтегрованої системи Coocox IDE.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Ознайомитися з структурою STM32F407, організацією паралельних портів, інтерфейсом I2C, Flash FM24CL16, організацією інтерфесу RS-485, керуванням світлодіодами та реле.
2. Перевірити свою теоретичну підготовку по контрольних питаннях для самоперевірки; при необхідності скористатися методичними матеріалами.
3. Освоїти основні етапи процесу створення проекту в системі mVision5 у відповідності до методичних матеріалів.
4. Освоїти основні етапи процесу створення проекту в середовищі Coocox IDE у відповідності до методичних матеріалів.
5. Завантажити тестову програму до лабораторної роботи № 5 в середовищі Coocox IDE у відповідності до методичних матеріалів.
6. Запустити та налаштувати програму ModbusFS 2.2.
7. Запустити необхідний тестовий проект в режимі покрокового виконання.
8. Захистити лабораторну роботу.

ЗАВДАННЯ

1. В покроковому режимі продемонструвати ініціалізацію мікроконтролера та відлагоджувальної плати.
2. В покроковому режимі продемонструвати ініціалізацію паралельних портів.
3. В покроковому режимі продемонструвати ініціалізацію I2C.
4. В покроковому режимі продемонструвати ініціалізацію RS-485 (UART).
5. В покроковому режимі продемонструвати роботу драйвера світлодіодів.
6. В покроковому режимі продемонструвати роботу драйвера реле.
7. В покроковому режимі продемонструвати роботу драйвера RS-485.
8. В покроковому режимі продемонструвати роботу драйвера FM24CL16.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Внутрішня структура мікроконтролера STM32F407, організація паралельних портів, організація інтерфесу RS-485, інтерфейсу I2C, Flash FM24CL16, керування світлодіодами та реле.
2. Програмна модель STM32F407.
3. Основні опції меню відлагоджувача mVision5 та Coocox IDE.

ОСНОВНІ ЕТАПИ ВИКОНАННЯ РОБОТИ

Створення проекту та налагодження програми в інтегрованій системі mVision5

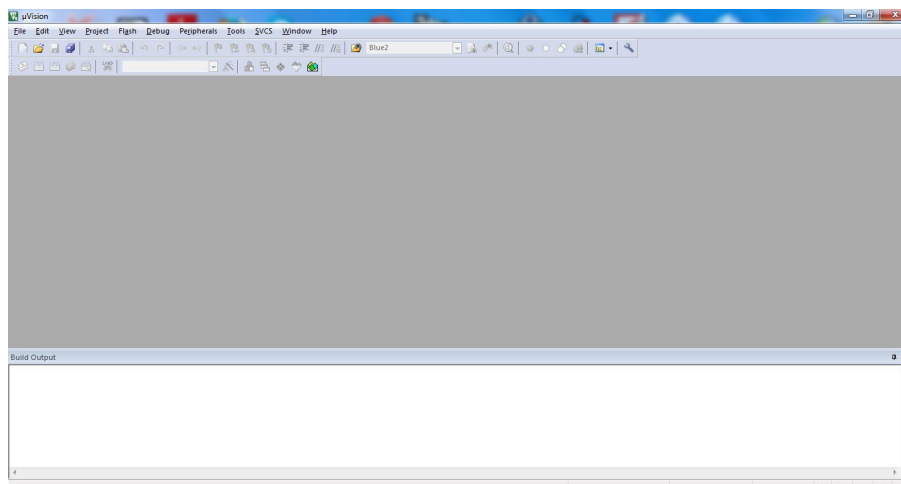


Рис.1. Стартове вікно mVision5

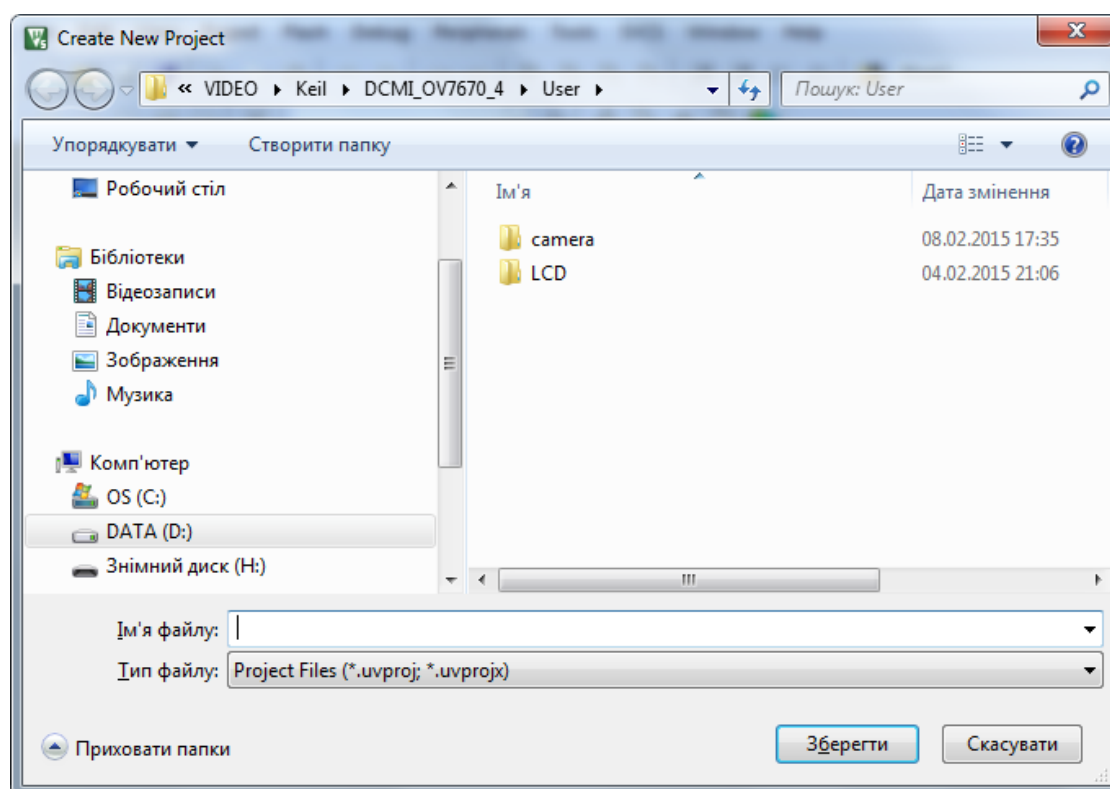


Рис.2. Створення нового проекту

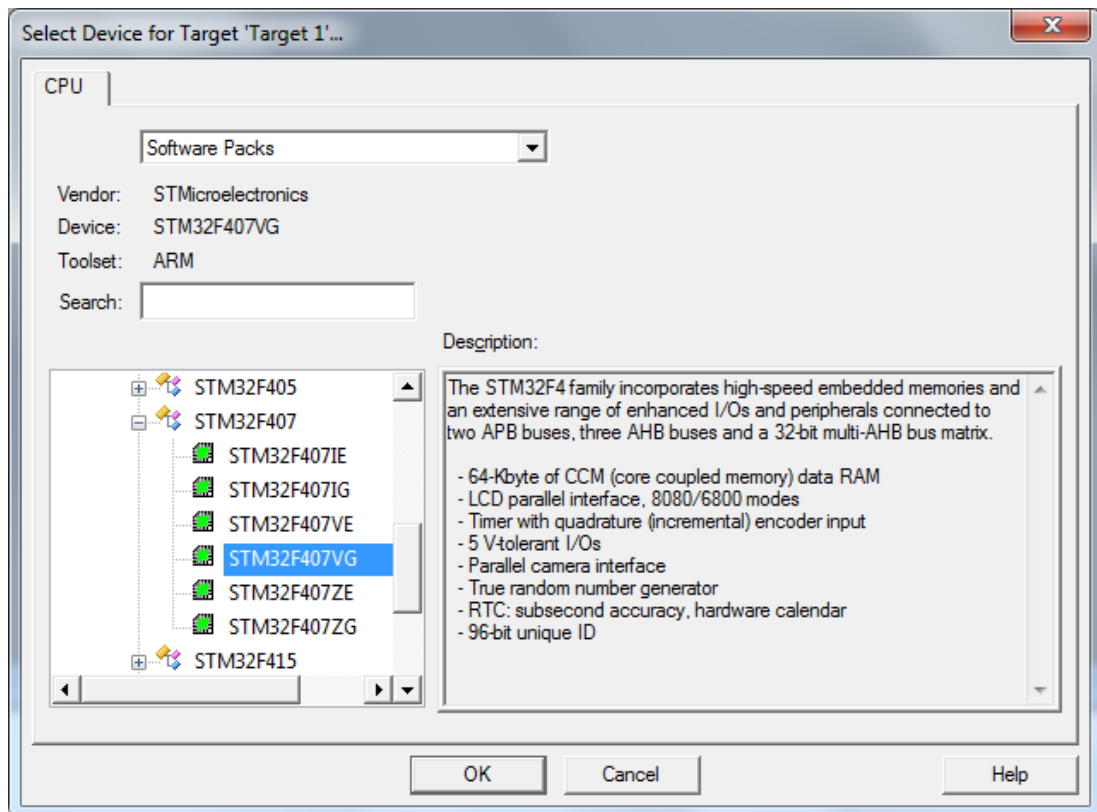


Рис.3. Вікно вибору мікропроцесора STM32407

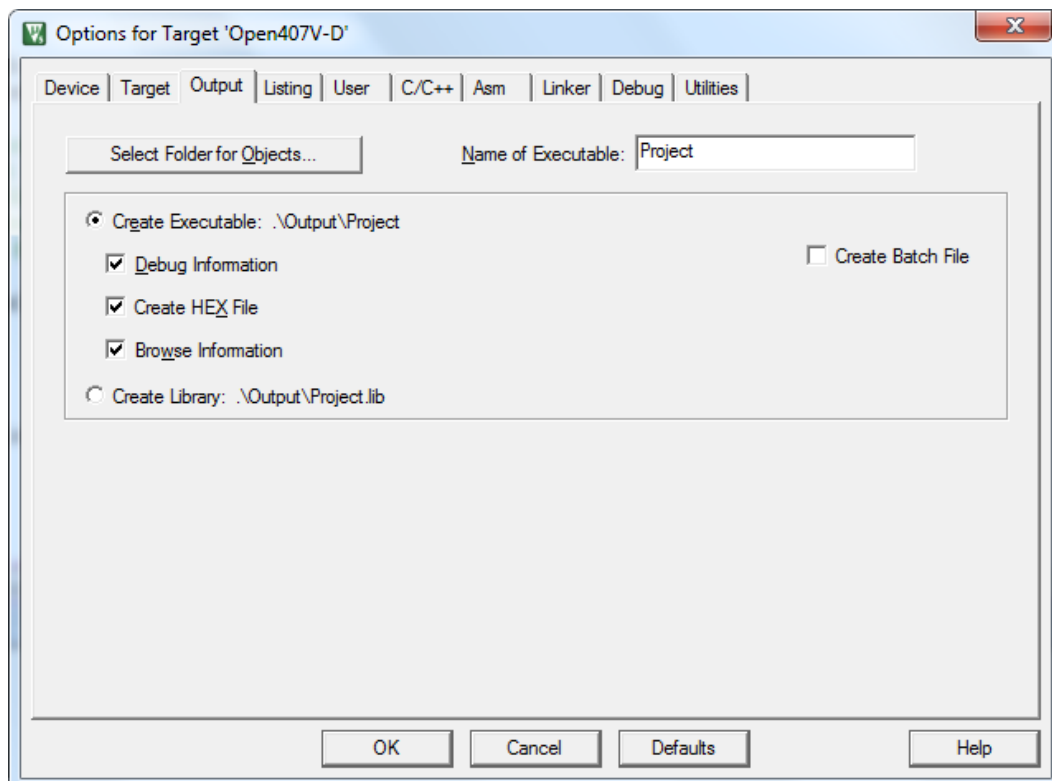


Рис.4. Вікно вибору для створення HEX-модуля

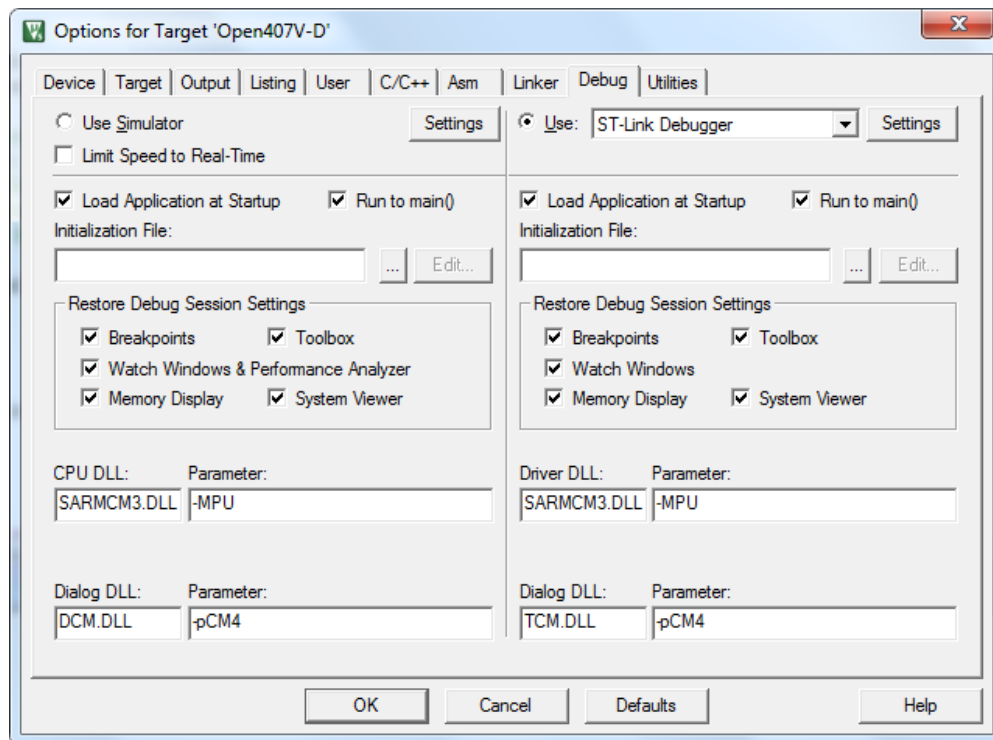


Рис.5. Вікно програми відлагоджувача проектів

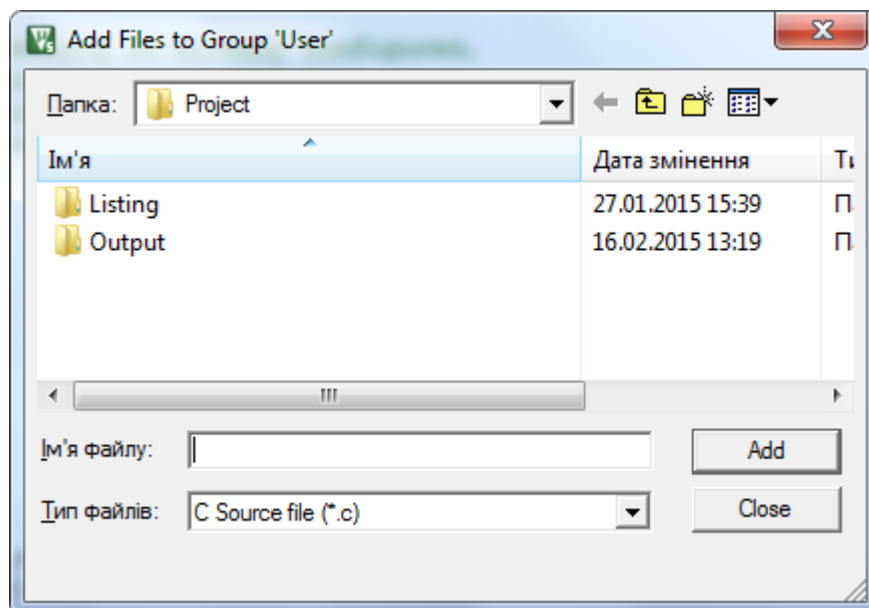


Рис.6. Вікно завантаження програми в проект

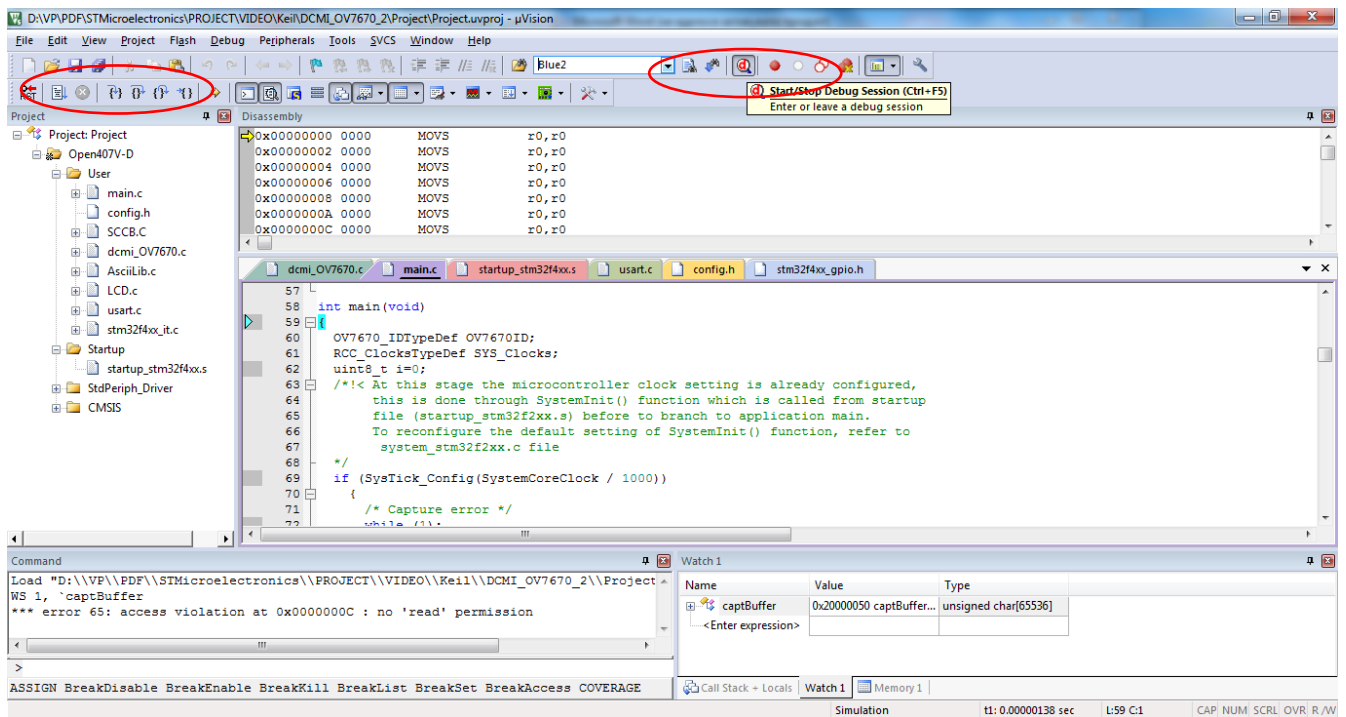


Рис.7. Вікно програми відлагоджувача проектів, програма завантажена та відкомпільована

Створення проекту з використанням Pack Installer

Запускаємо Keil 5 та Pack Installer.

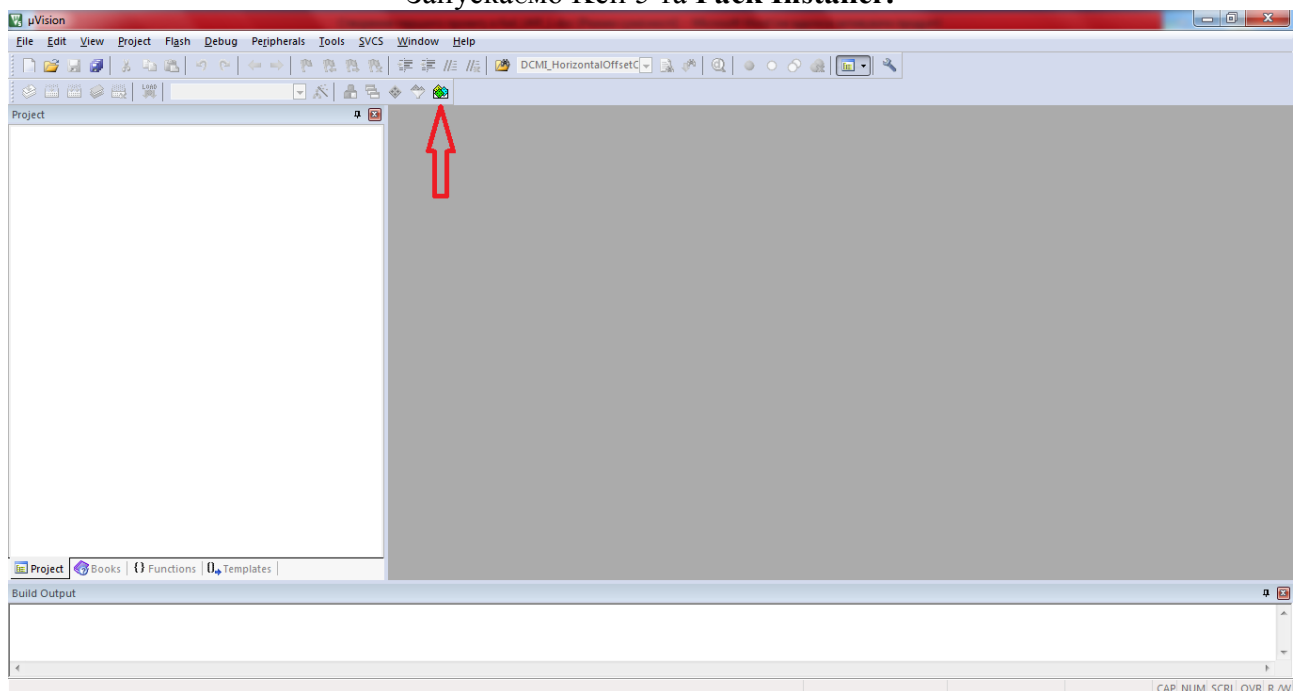


Рис.8. Запуск Pack Installer

На панелі інструментів вгорі натискаємо зазначену кнопку. Відкривається вікно **Pack Installer** в ньому дві вкладки зліва і дві справа

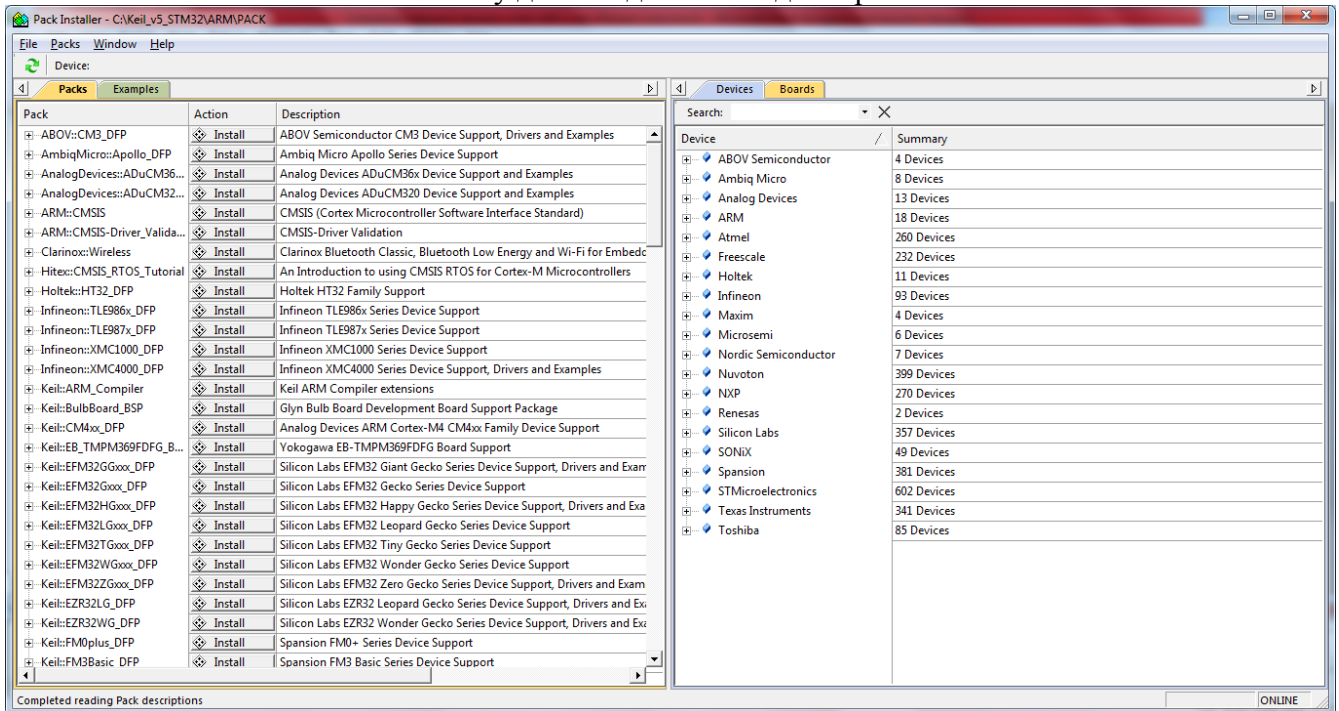


Рис.9. Вибір сімейства мікроконтролера

Натискаємо на вкладку **Devices** вибираємо зі списку виробників STMicroelectronics далі вибираємо серію МК (МК- мікроконтролер) STM та тип МК STM32F407VG. І на вкладці **Pack** по черзі інсталуємо всі пакети.

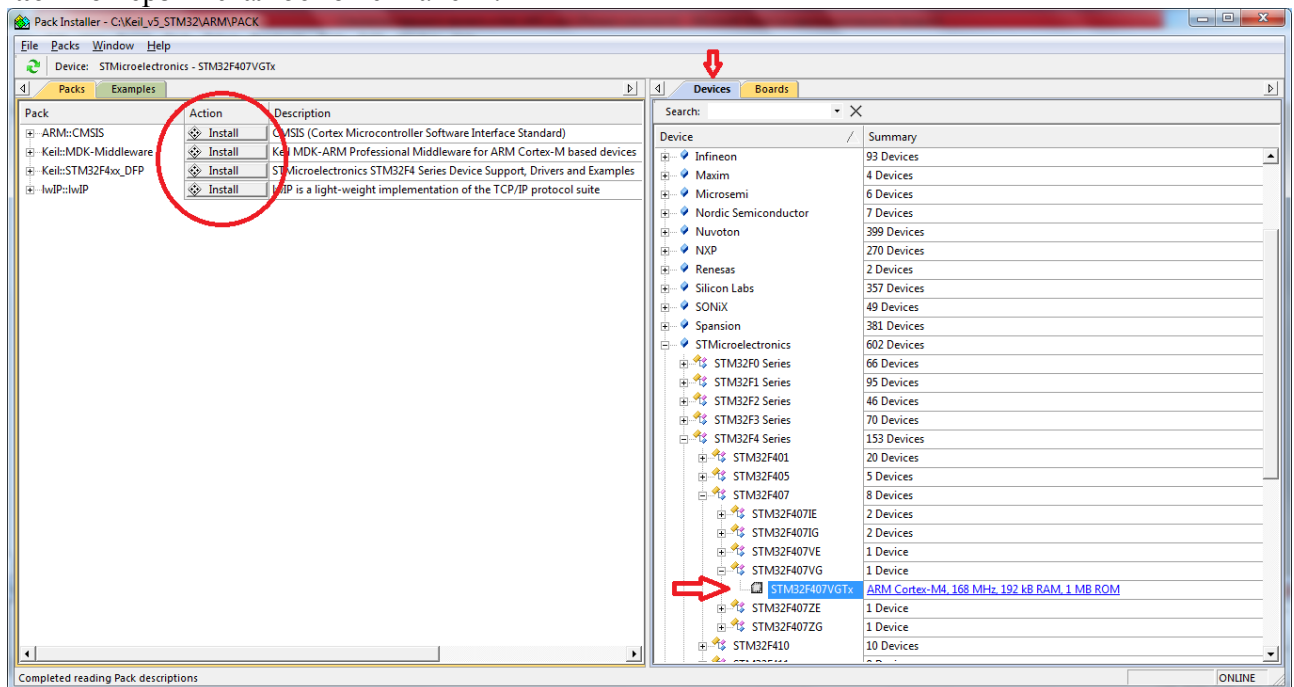


Рис.10. Вибір типу мікроконтролера та інсталяція модулів

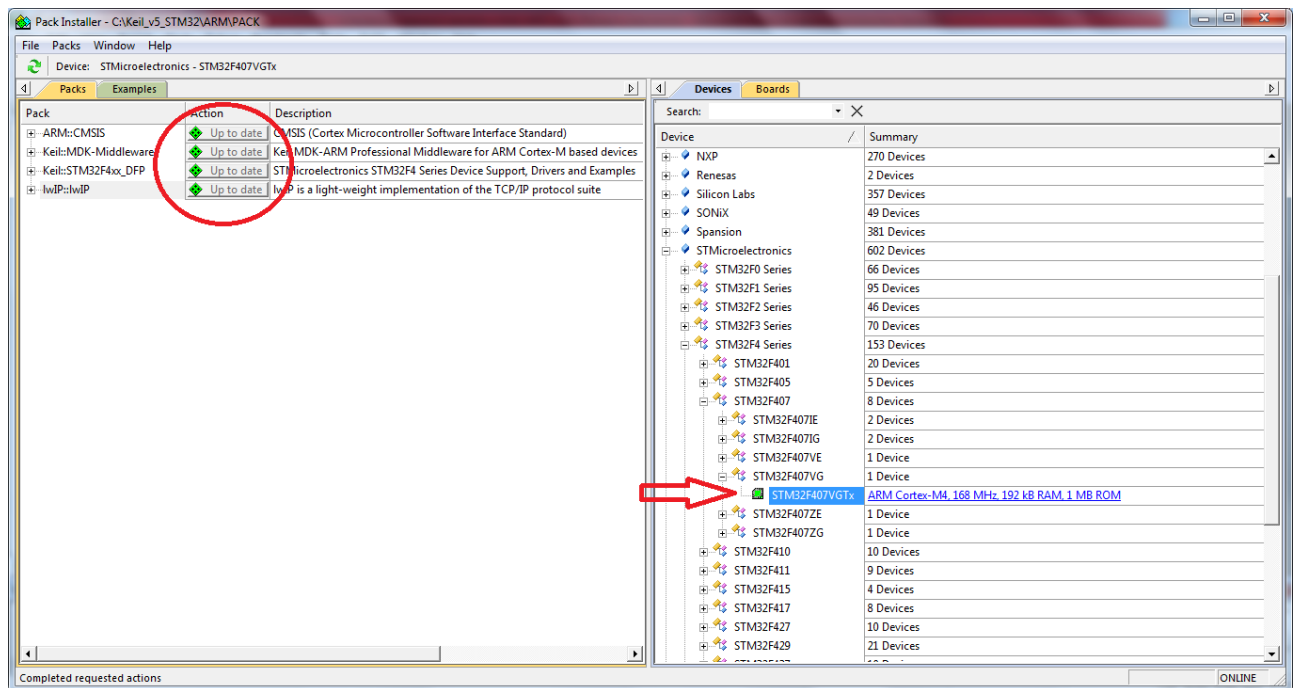


Рис.11. Результат інсталяції модулів

Далі створюємо проект: натискаємо кнопку **Project** і у випадяючому меню натискаємо **NewProject**

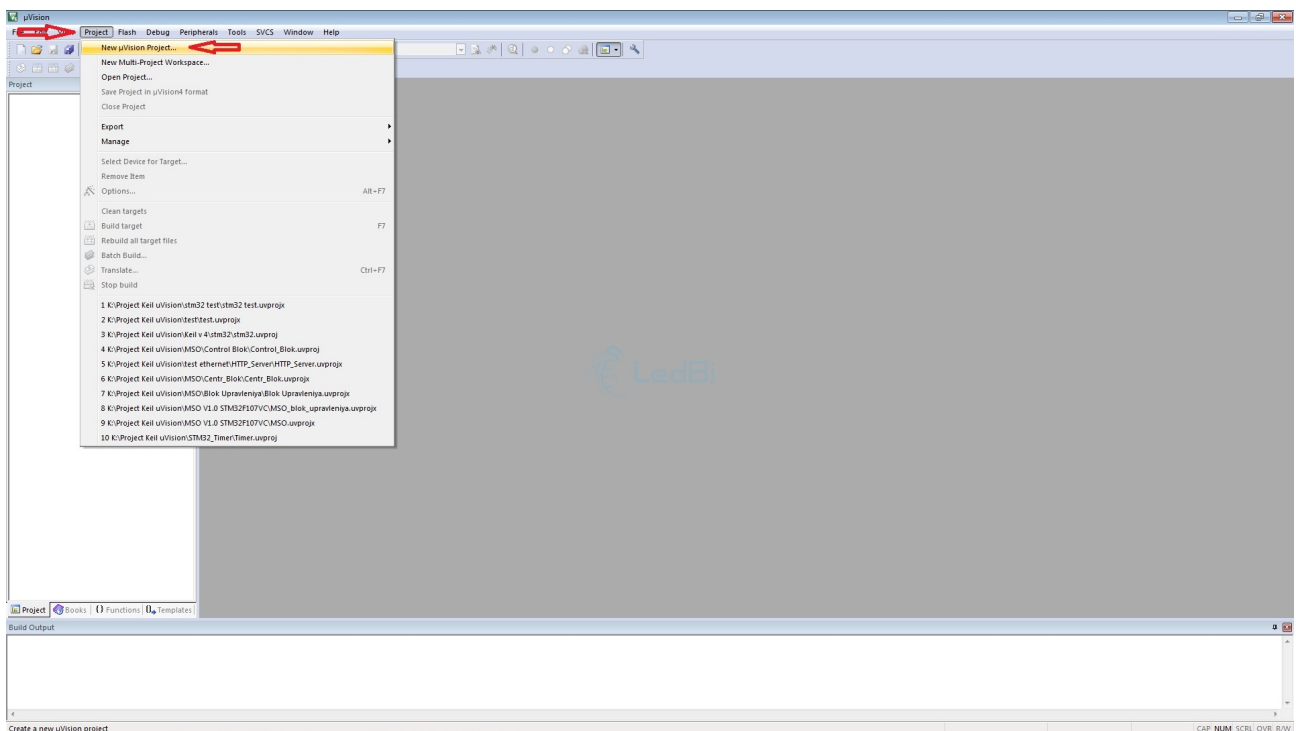


Рис.12. Створення проекту

З'являється вікно і пропонує вибрати де буде зберігатися проект. Далі з'явиться вікно і пропонує вибрати мікроконтролер.

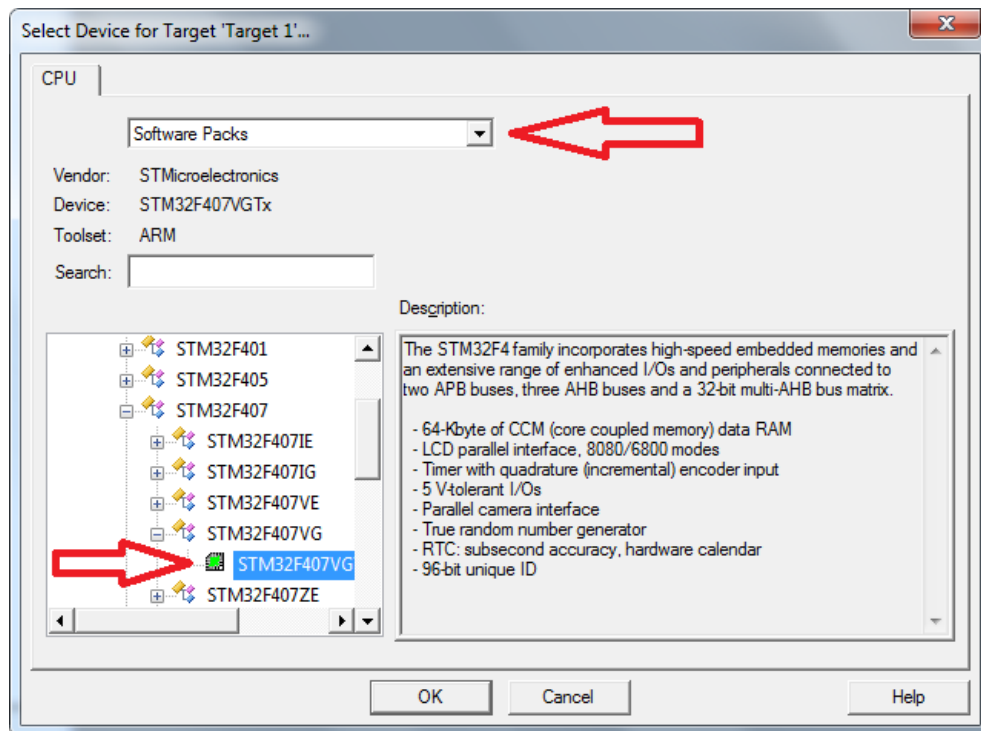


Рис.13. Вибір типу мікроконтролера

Наступне вікно **Manage Components**. Ставимо галочки згідно рисунків:

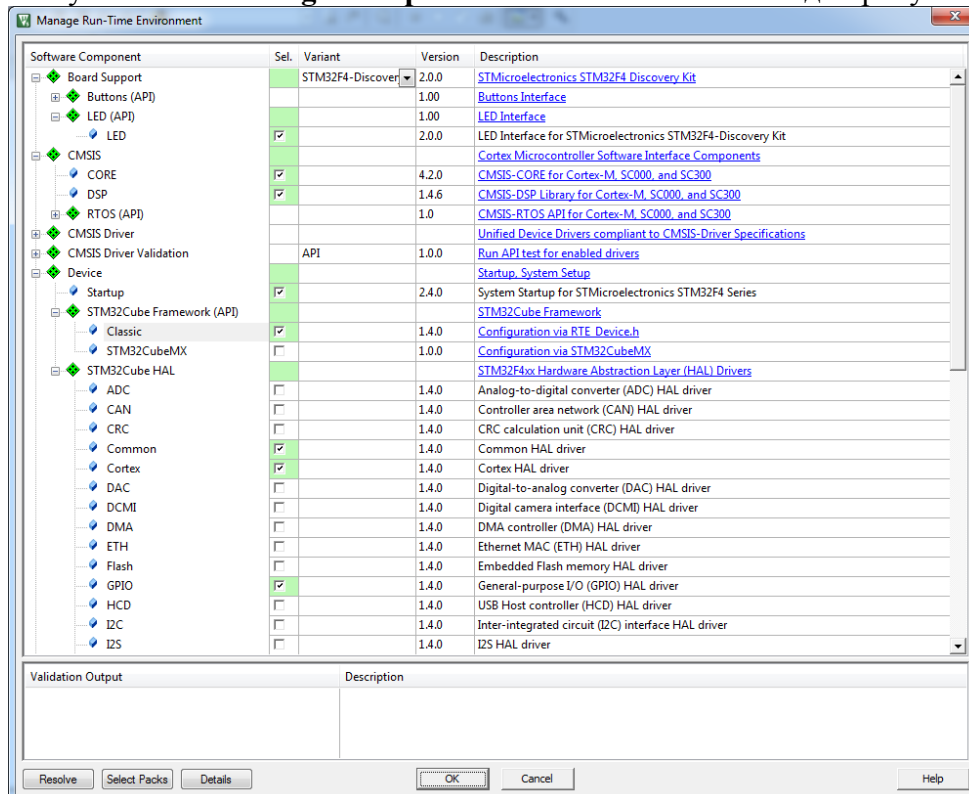


Рис.14. Вибір модулів

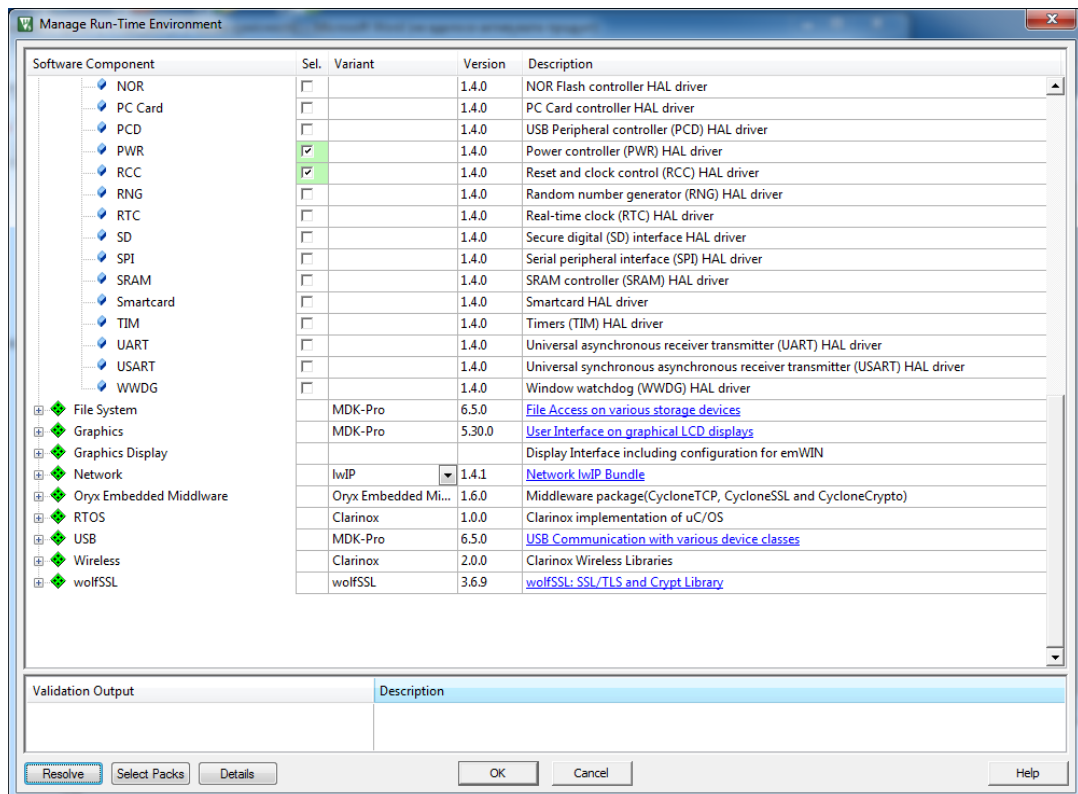


Рис.14а. Вибір модулів

Рядок "**Board Support**" відкриває список підтримуваних налагоджувальних плат, якщо там є ваша то можете вибрати її і поставити галочку Keil завантажить для неї драйвер.
"CMSIS" - тут ставимо галочку **"CORE"** це підтримка основного ядра ARM.
"RTOS API" - це операційна система реального часу.
CMSIS DRIVER - це драйвера інтерфейсів.

DEVICE - тут міститься практично вся основна периферія мікроконтролера.

Ставимо галочки **GPIO** - це основний драйвер портів введення/виводу,

Startup - це основний конфігураційний системний файл.

Ставимо галочки і натискаємо **OK**. Якщо у вас квадратики світяться жовтим кольором – натискаєте в самому низу вікна кнопку **"Resolve"** і якщо відповідні драйвери інстальовано то вони стануть зеленими, інакше необхідно доінстальювати відповідні драйвери.

В меню **DEVICE** вибираєте **GPIO** і **Startup**.

Далі в **DEVICE** ставите галочки на тій периферії яка вам потрібна і натискаєте в самому низу вікна кнопку **"Resolve"** потім **OK** Вікно повинне зникнути. Далше треба створити файл для коду. Як показано на рисунку правою кнопкою миші по **Source_group 1** далі **Add new item..**. Далі вибираємо тип файлу. І пишемо його ім'я в полі **"Name"**, наприклад, **main**, і натискаємо **ADD**.

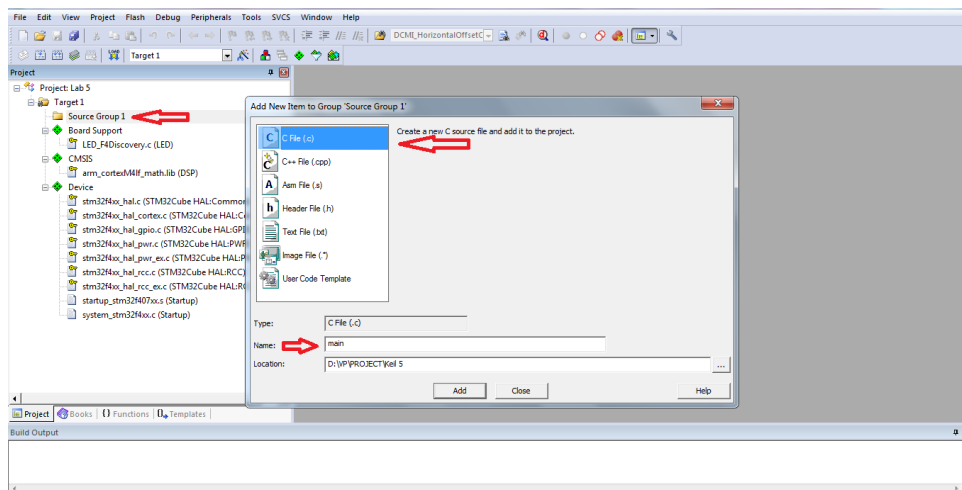


Рис.15. Вибір типу файла

Натискаємо правою кнопкою миші в полі де повинен бути код і вибираємо Insert #include file далі **stm32f4xx.h**. Якщо у вас інший мікроконтролер виконуєте все те ж саме тільки замість **stm32f4xx.h** обираєте свій файл.

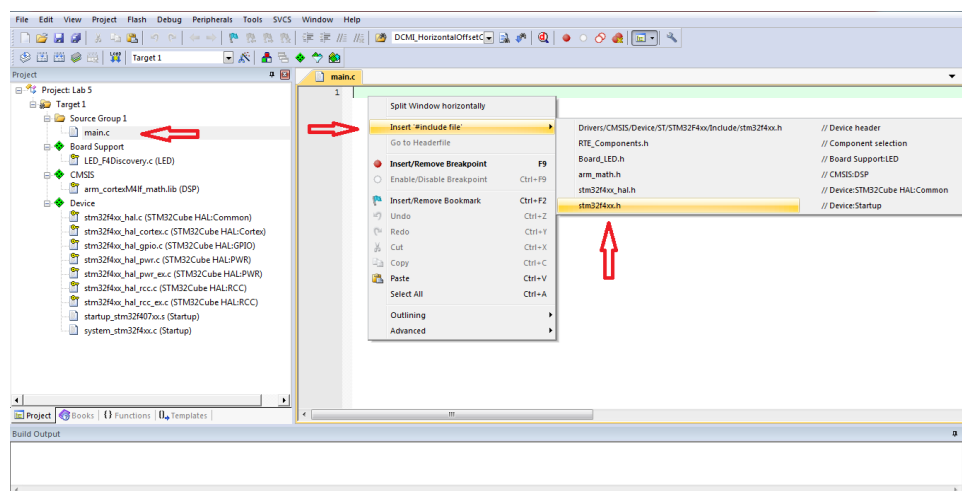


Рис.16. Підключення файлів типу *.h

Далі на панелі інструментів натискаємо кнопку "Options for Target"

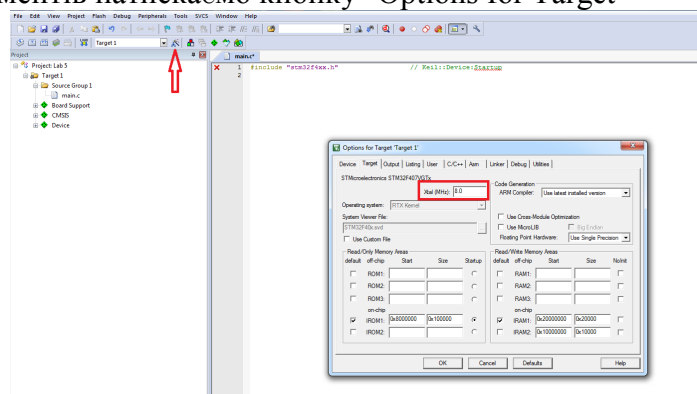


Рис.17. Вибір частоти тактового генератора

Виставляємо частоту Xtal. В наступній вкладці вибрати формування вихідного файлу *.hex :
“Create HEX file”

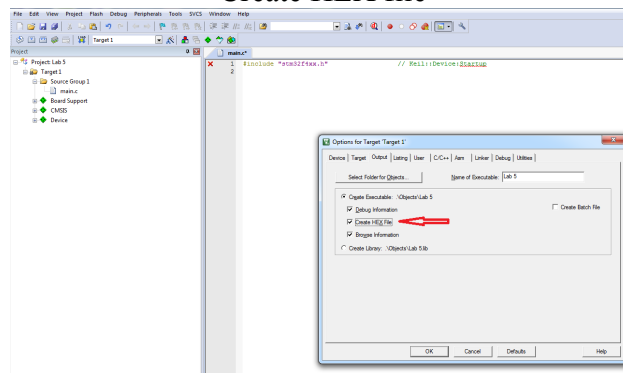


Рис.18. Вибір формування вихідного файлу *.hex

Відкриваємо «main» і пишемо програму.

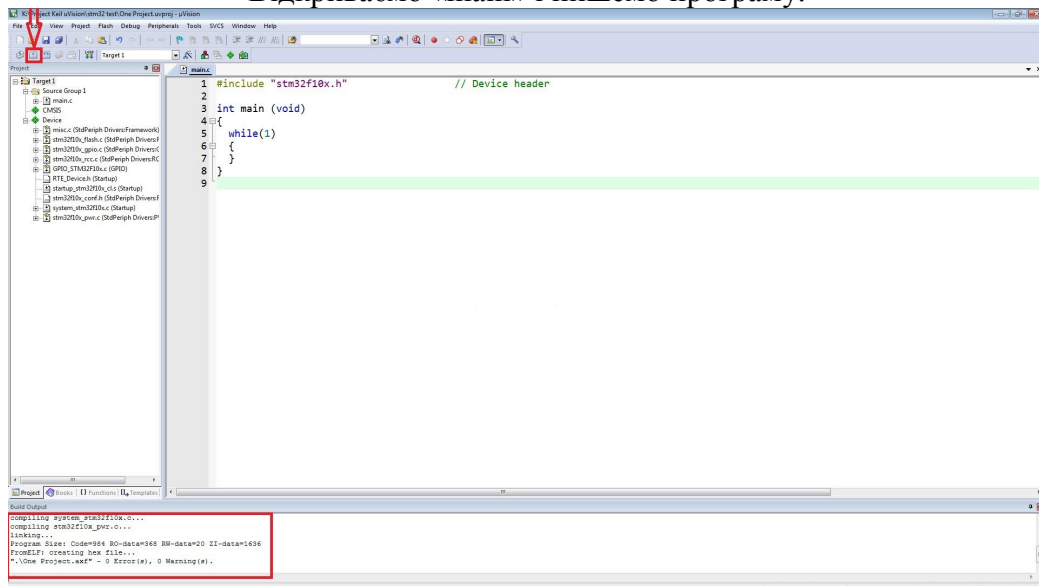


Рис.19. Підготовка та компіляція програми «main»

На панелі інструментів натискаємо кнопку “Build”.

ВІДЛАГОДЖЕННЯ ПРОГРАМИ в середовищі Keil 5

Відлагоджувати програму можна з використанням симулятора («Use Simulator») або апаратно-програмного відлагоджувача, наприклад, «ST-Link Debugger», див. рис.22.

Для відлагодження в реальному часі з використанням апаратної платформи заданого мікроконтролера Keil uVision5 забезпечує підтримку широкого набору адаптерів. Крім цього в середовищі є вбудований програматор, який забезпечує програмування пам'яті програм та даних, див. рис.24.

Модуль STM32F4DISCOVERY. Цей модуль спроектовано для відлагодження апаратних вузлів мікропроцесорної системи на базі мікроконтролерів STM32F407/417. На платі модуля розміщено мікроконтролер STM32F407VGT6 в корпусі LQFP100 з вузлом синхронізації на базі кварцевого резонатора 8MHz, вузол USB, аудіо-ЦАП, 4 світлодіоди користувача, кнопка RESET, кнопка користувача та контактні групи підключені до виводів

мікроконтролера. Крім цього на платі розміщено адаптер ST-LINK/V2 який використовується для відлагодження в реальному часі та програмування пам'яті Flash мікроконтролера. Адаптер може бути відключений від мікроконтролера на платі та використовуватися для відлагодження та програмування плати користувача через інтерфейс SWD.



Рис. 20. Зовнішній вигляд модуля STM32F4DISCOVERY

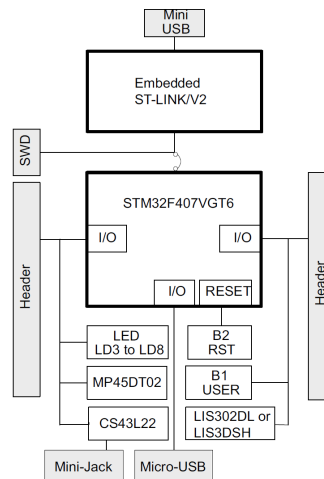


Рис. 21. Структура модуля STM32F4DISCOVERY

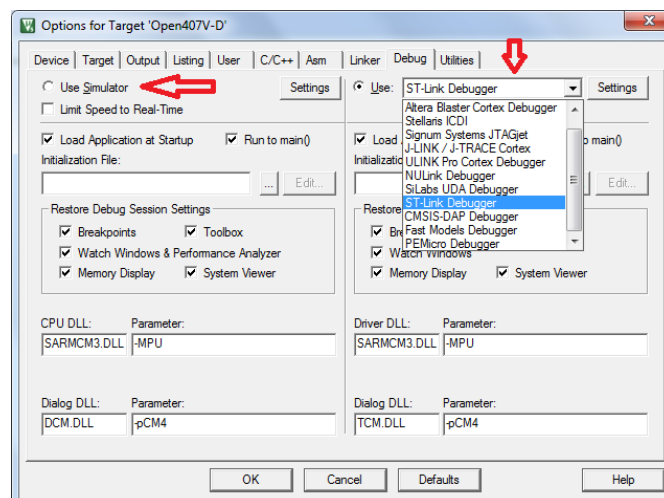


Рис.22. Вікно вибору адаптера для програмування та відлагодження в Keil uVision5

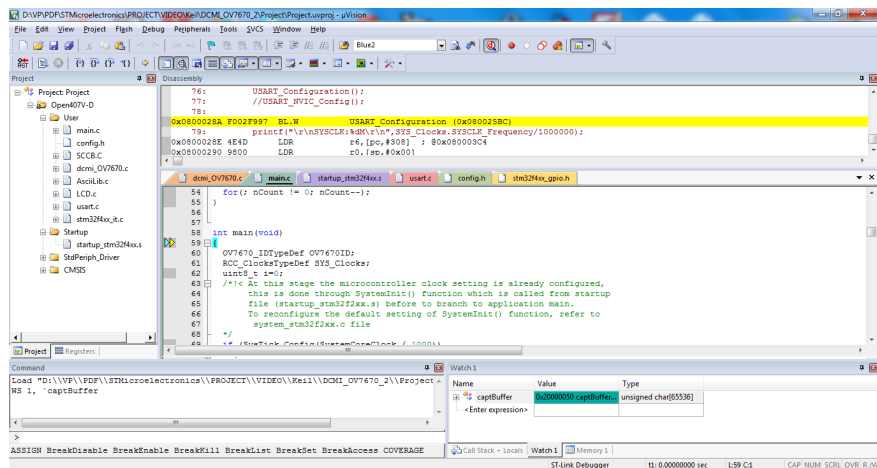


Рис.23. Вікно відлагодження програми в Keil uVision5

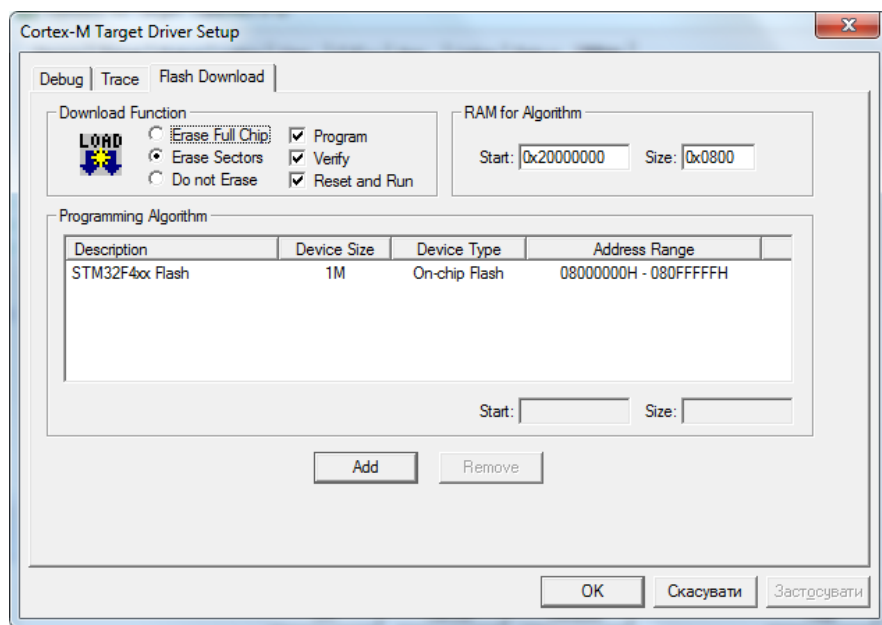


Рис.24. Вікно конфігурації вбудованого програматора в Keil uVision5

Створення проекту та відлагодження програми в інтегрованій системі CooCox IDE

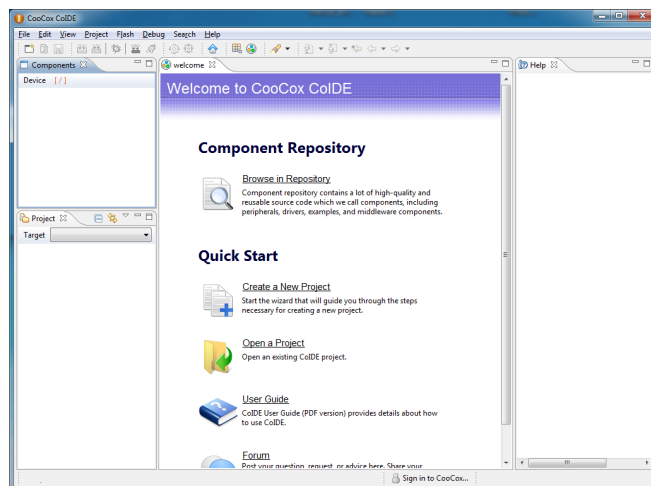


Рис.25. Стартове вікно CooCox IDE

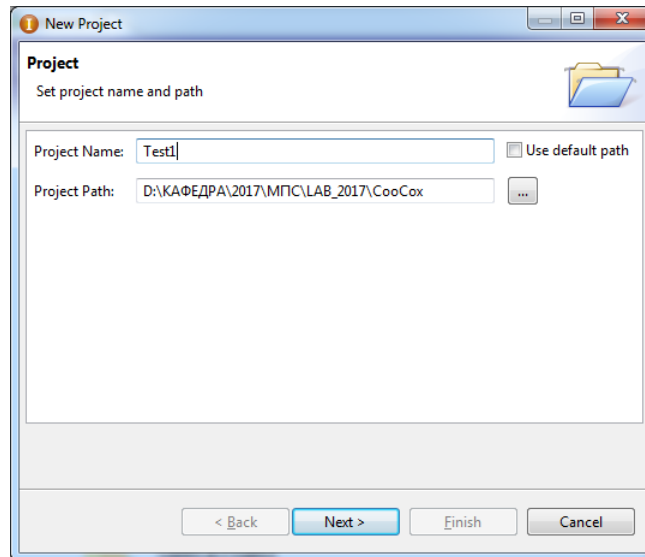


Рис.26. Створення проекту

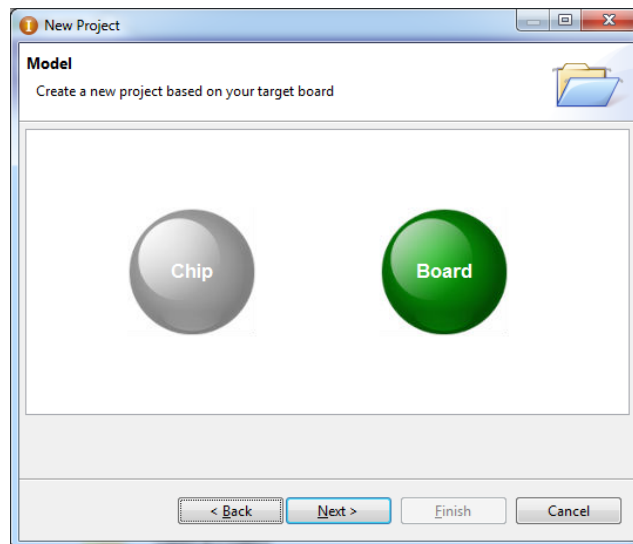


Рис.26. Вибір типу мікроконтролера або відлагоджувального модуля

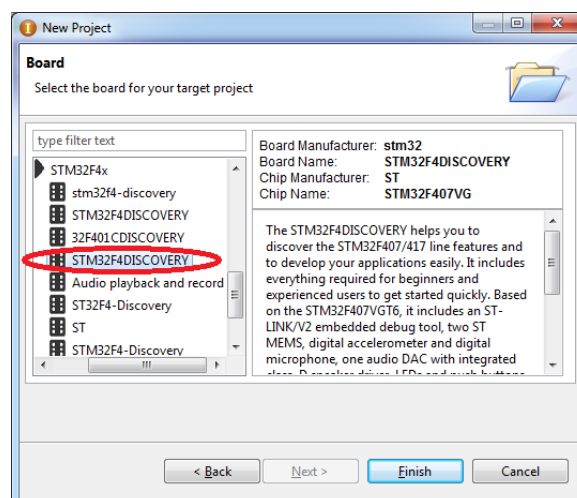


Рис.27. Вибір типу відлагоджувального модуля STM32F407DISCOVERY

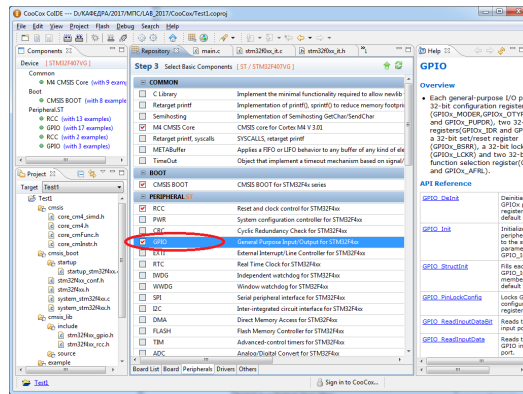


Рис.28. Вибір тестового проекту для модуля STM32F407DISCOVERY

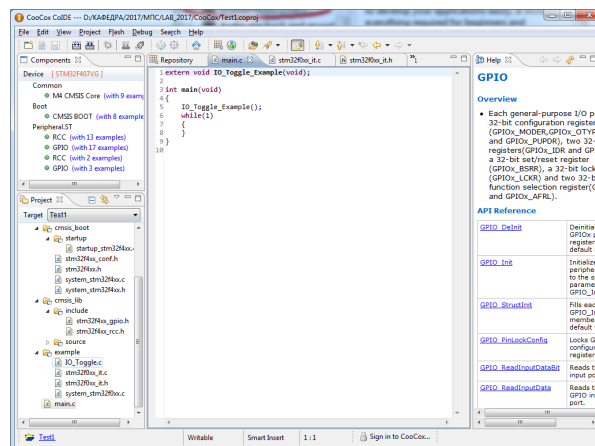


Рис.29. Вікно модуля «main»

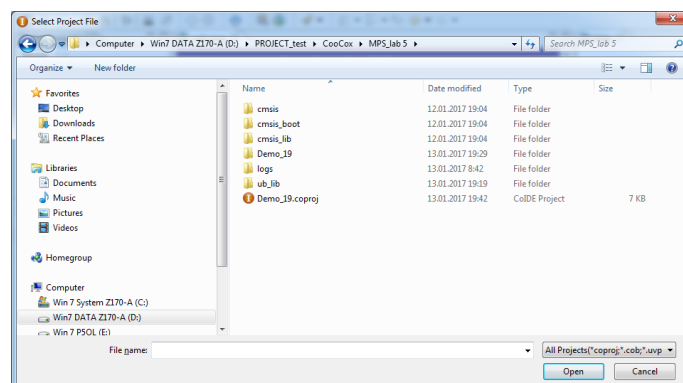


Рис.30. Завантаження готового тестового проекту в Coocox

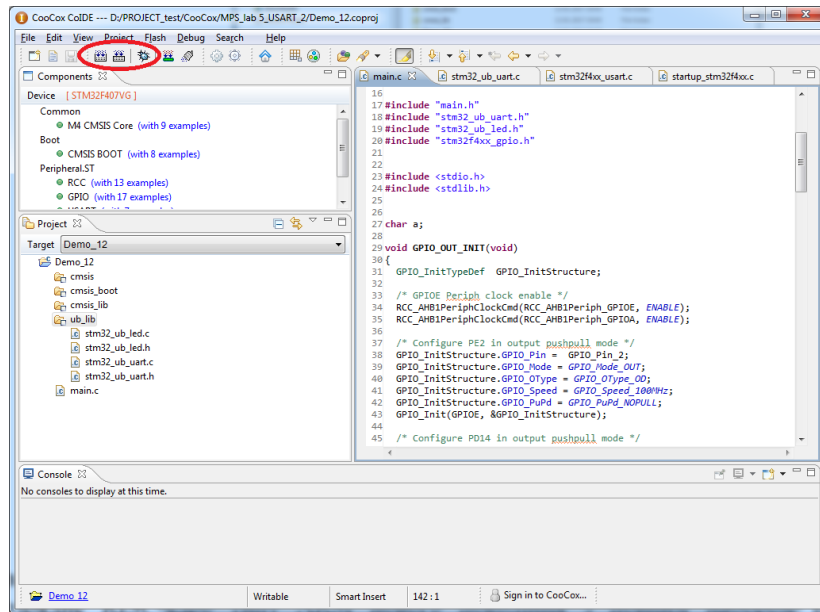


Рис.31. Компіляція та запуск режиму відлагодження тестового проекту в Coocox

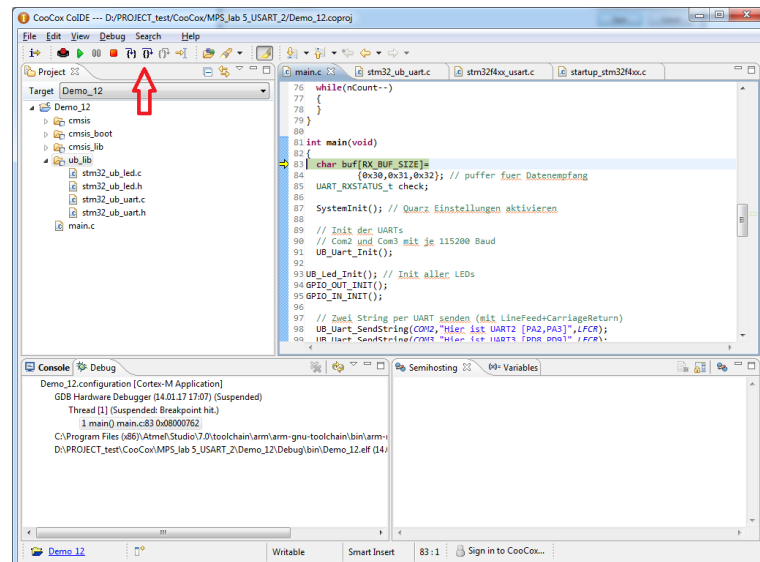


Рис.31. Покрокове виконання тестового проекту Coocox керування світлодіодами, реле та передачі даних по RS-485

Прийом інформації по RS-485 в ПК здійснюється програмою «Modbus FS».

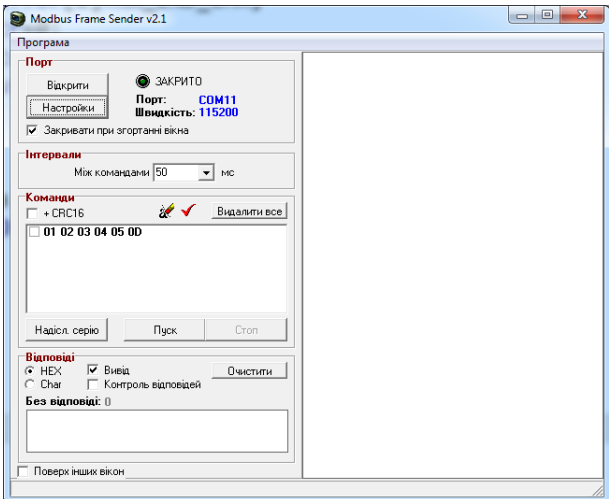


Рис.32. Стартове вікно програми «Modbus FS»

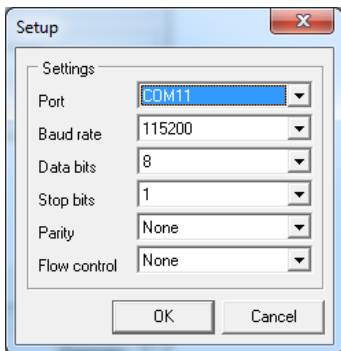


Рис.33. Налаштування програми «Modbus FS»

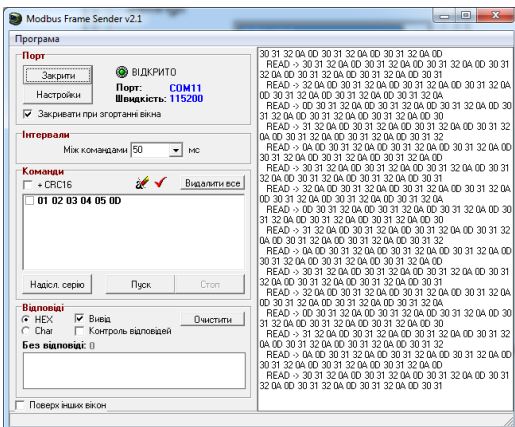


Рис.34. Вікно виводу програми «Modbus FS»

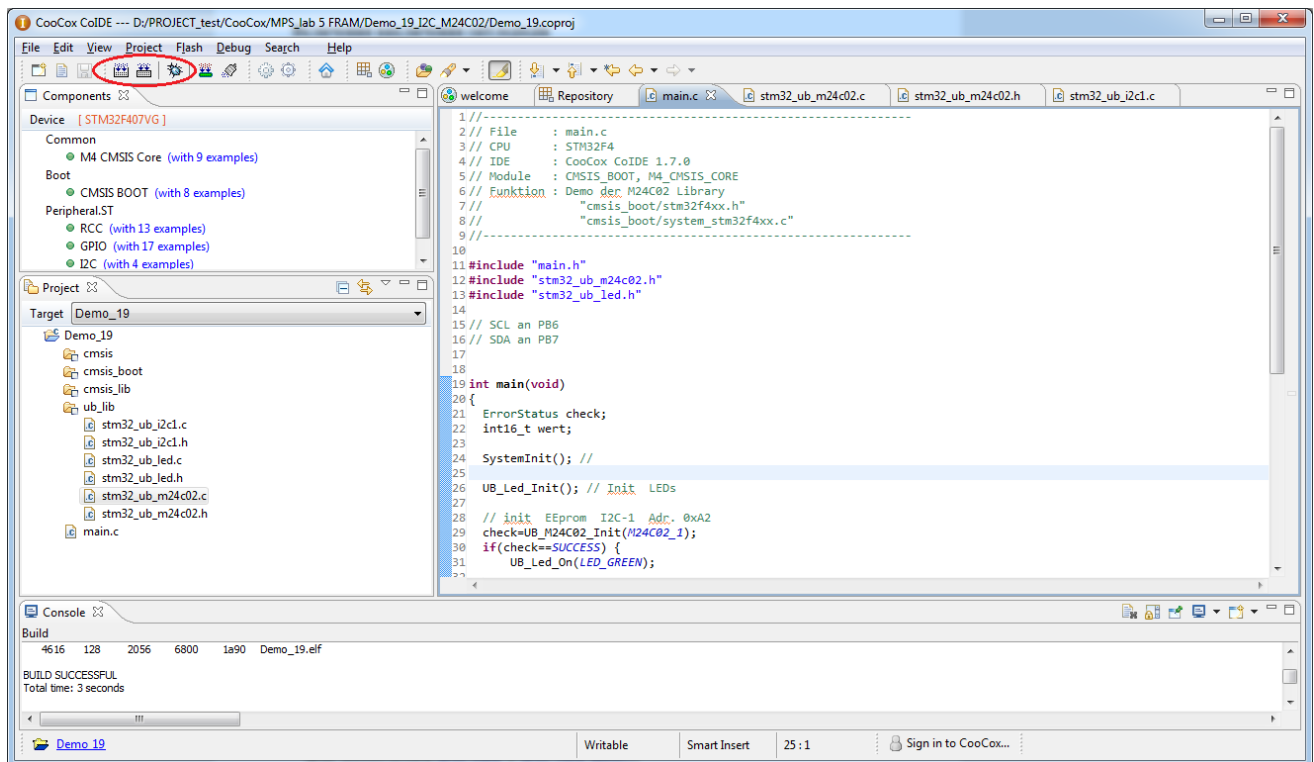


Рис.35. Компіляція та запуск режиму відлагодження тестового проекту Coocox звертання до FM24CL16

МЕТОДИЧНІ МАТЕРІАЛИ

Ядро Cortex-M4F розроблено для проектування вбудованих мікрокомп'ютерних систем середньої продуктивності з низькою споживною потужністю. Архітектурно це 32-розрядне RISC-ядро типу ARM нового покоління з повним набором інструкцій ARM, реалізацією інструкцій DSP, вбудованим процесором плаваючої коми (FPU) та максимальною робочою частотою до 200 MHz. Ядро має вбудований модуль захисту пам'яті який підвищує безпеку роботи виконання програм. Для оптимального програмування FPU використовується спеціальна мета-мова. Все це дозволяє ефективно обробляти сигнали та реалізовувати складні алгоритми керування.

На базі ядра Cortex-M4F різні фірми освоїли випуск широкої номенклатури мікроконтролерів з різними об'ємами інтегрованої на кристалі пам'яті та різноманітною номенклатурою інтегрованих периферійних пристроїв. Одними з таких мікроконтролерів є мікроконтролери серії STM32F4xx фірми STMicroelectronics які стали досить популярними серед розробників.

Вказані мікроконтролери характеризуються наявністю інтегрованої швидкодіючої пам'яті Flash до 1 Mb та наявністю кеш-пам'яті що забезпечує звертання до пам'яті на максимальній частоті процесора без циклів очікування. Система обробки подій в реальному часі не вимагає втручання процесора що забезпечує обробку без затримок. Номенклатура периферійних пристроїв різних мікроконтролерів поряд з класичними інтерфейсними пристроями включає такі комунікаційні інтерфейси як Ethernet MAC з підтримкою стандарту IEEE1588, CAN, USB, інтерфейси для підключення відеокамери та графічних дисплеїв, 12 та 16-розрядні багатоканальні швидкодіючі аналого-цифрові перетворювачі (АЦП), спеціалізовані цифро-аналогові перетворювачі (ЦАП). Для забезпечення високої продуктивності при обміні даними на кристалах інтегровані канали прямого доступу до пам'яті. Деякі мікроконтролери мають інтегровані на кристалі такі специфічні вузли як

криптографічний процесор, модуль обчислення контрольних сум CRC, генератор випадкових чисел.

Важливою особливістю вказаних мікроконтролерів є можливість функціонування в режимі пониженого споживання при батарейному живленні. Також при пропаданні основного живлення забезпечується зберігання важливих даних в SRAM з батарейним живленням та функціонування таймера реального часу. Мікроконтролери випускаються в різних корпусах з різною кількістю виводів від 64 до 176, що дозволяє оптимізувати апаратні засоби при різних застосуваннях. Напруга живлення мікроконтролерів від 1.8V до 3.6V, температура в робочому режимі від - 40 до +105 градусів С.

ВНУТРІШНЯ СТРУКТУРА STM32407.

Основні характеристики

- **ядро 32 розряди;**
- **максимальна частота ядра 168 MHz;**
- **вбудований процесор плаваючої коми FPU;**
- **інструкції ARM та DSP;**
- **вбудована Flash пам'ять програм до 1Mb та SRAM даних до 192 Kb з можливістю зовнішнього розширення;**
- **SRAM 4Kb з зовн. резервним живленням;**
- **широка номенклатура інтегрованої периферії;**
- **VDD = 3.3 V ($1.8\text{ V} \leq \text{VDD} \leq 3.6\text{ V}$)**

Вузли STM32F40x

- **ядро ARM® Cortex™-M4F 168 MHz;**
- **система синхронізації PLL;**
- **інтегрований процесор FPU;**
- **інтегрована Flash пам'ять програм до 1Mb;**
- **інтегрована SRAM даних до 192 Kb;**
- **інтегрована резервна SRAM даних 4Kb (Vbat);**
- **адаптивний акселератор пам'яті (ART Accelerator);**
- **блок захисту пам'яті (MPU);**
- **модуль обчислення контрольних сумм (CRC unit);**
- **генератор випадкових чисел (RNG);**
- **мульти-АНВ матриця шин 32 р;**
- **контролер прямого доступу до пам'яті (DMA);**
- **контролер статичної пам'яті (FSMC) з можливістю конфігурації керування PKI;**
- **контролер вкладених векторних переривань (NVIC);**
- **контролер зовнішніх переривань (EXTI);**
- **широка номенклатура інтерфейсних вузлів для паралельного та послідовного обміну (GPIO, (SDIO)-SD/SDIO/MMC, USART, SPI, I²C, CAN, USB, Ethernet, I2S);**
- **інтерфейс цифрової камери (DCMI);**
- **інтегровані 12р АЦП та ЦАП;**
- **універсальні таймери, RTC, WDT, SysTick ;**
- **вбудовані макро-регістри трасування (Embedded Trace Macrocell, ETM);**
- **JTAG та SW інтерфейси для трасування та програмування інтегрованої Flash;**
- **інтегрований стабілізатор напруги живлення;**
- **вузол керування живленням ;**

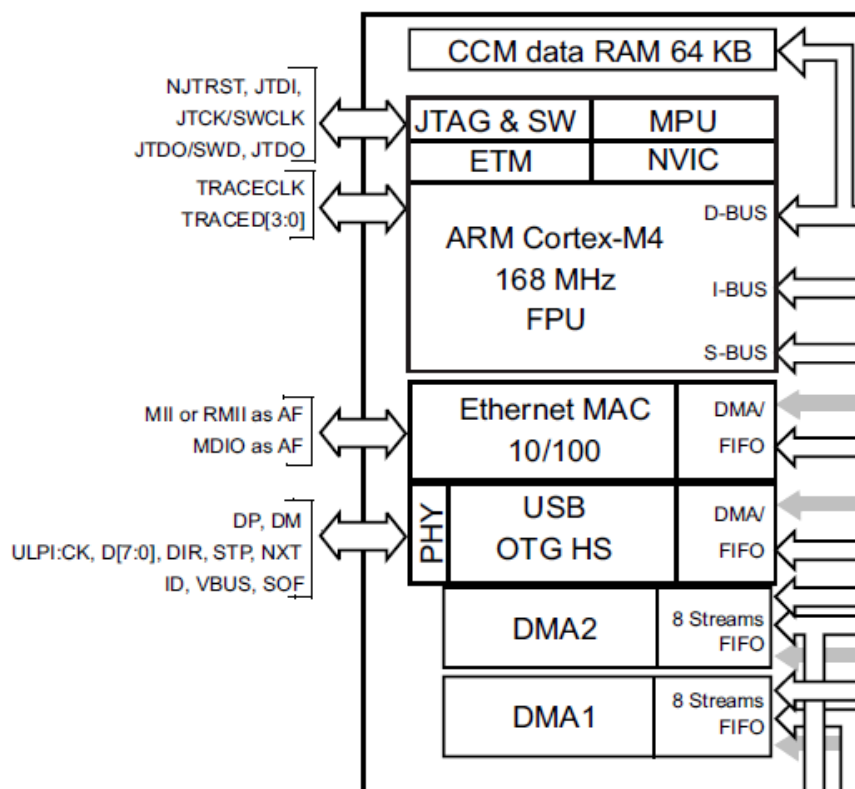
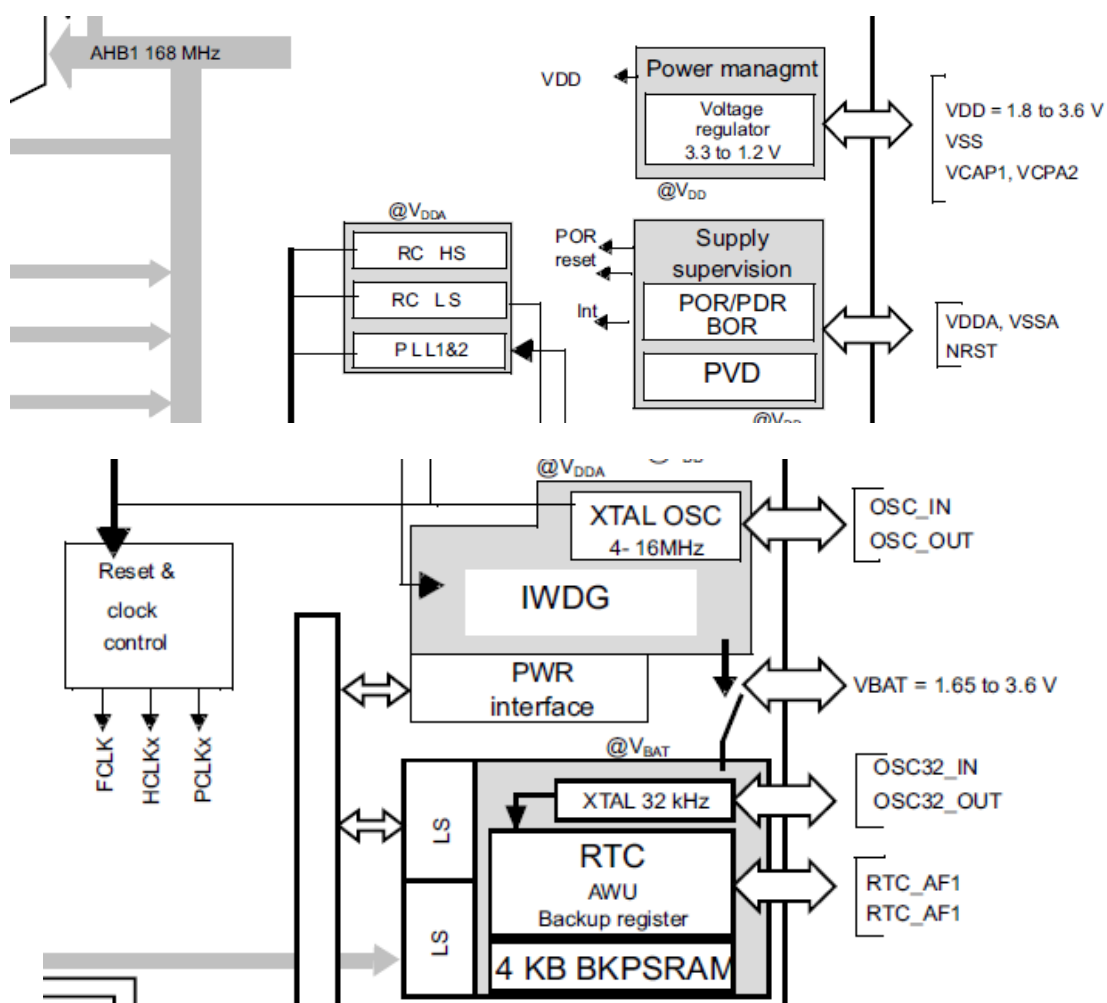


Рис.36а. Внутрішня структура STM32F407



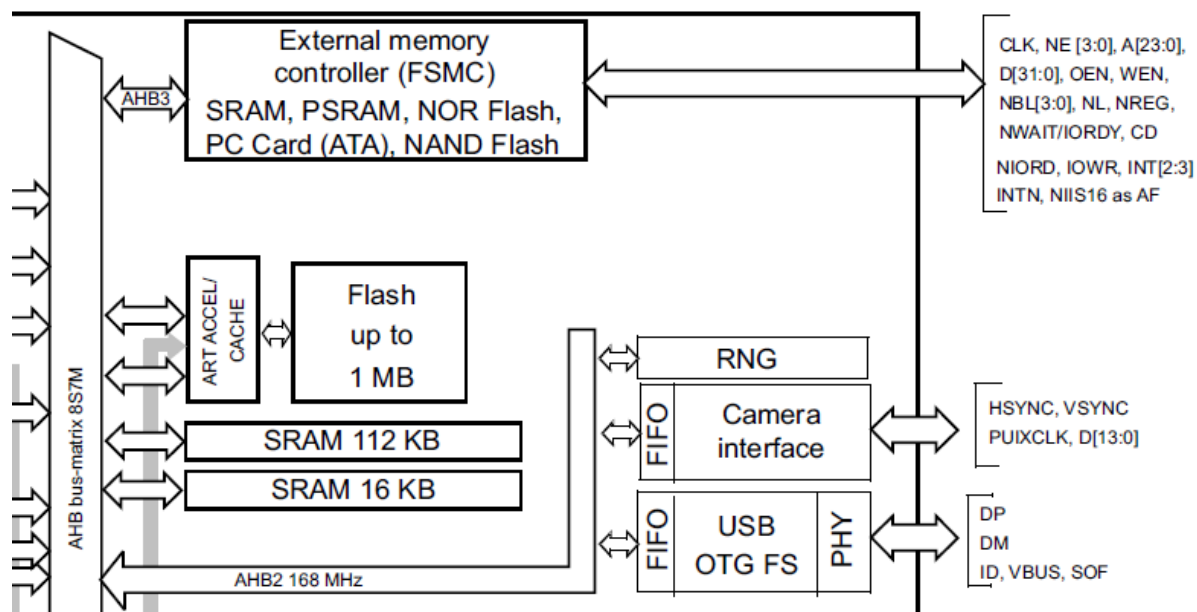
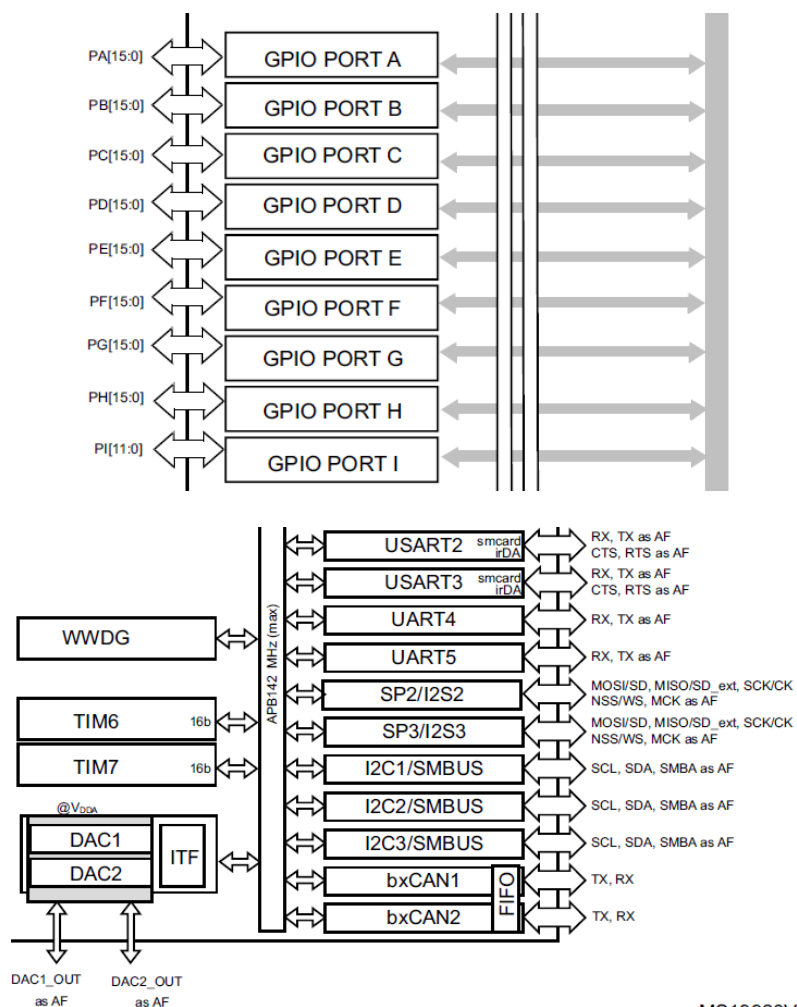


Рис.366. Внутрішня структура STM32F407



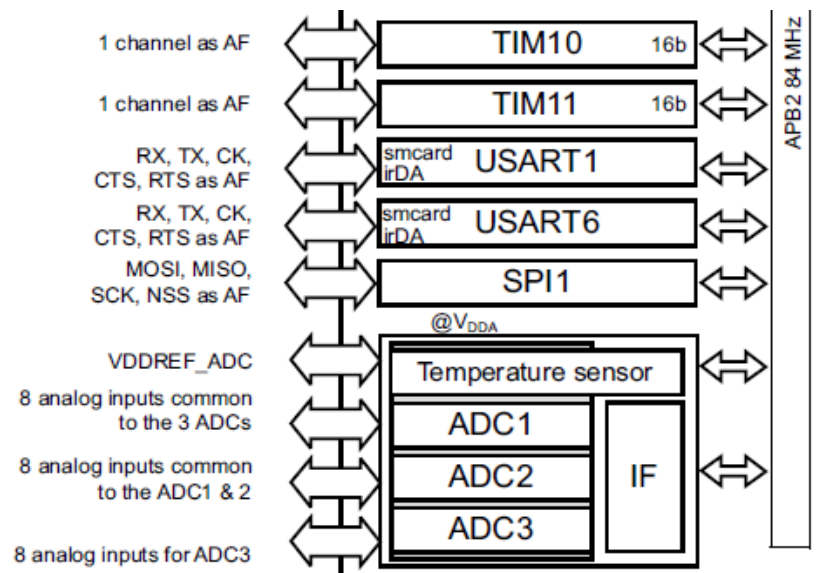


Рис.36в. Внутрішня структура STM32F407

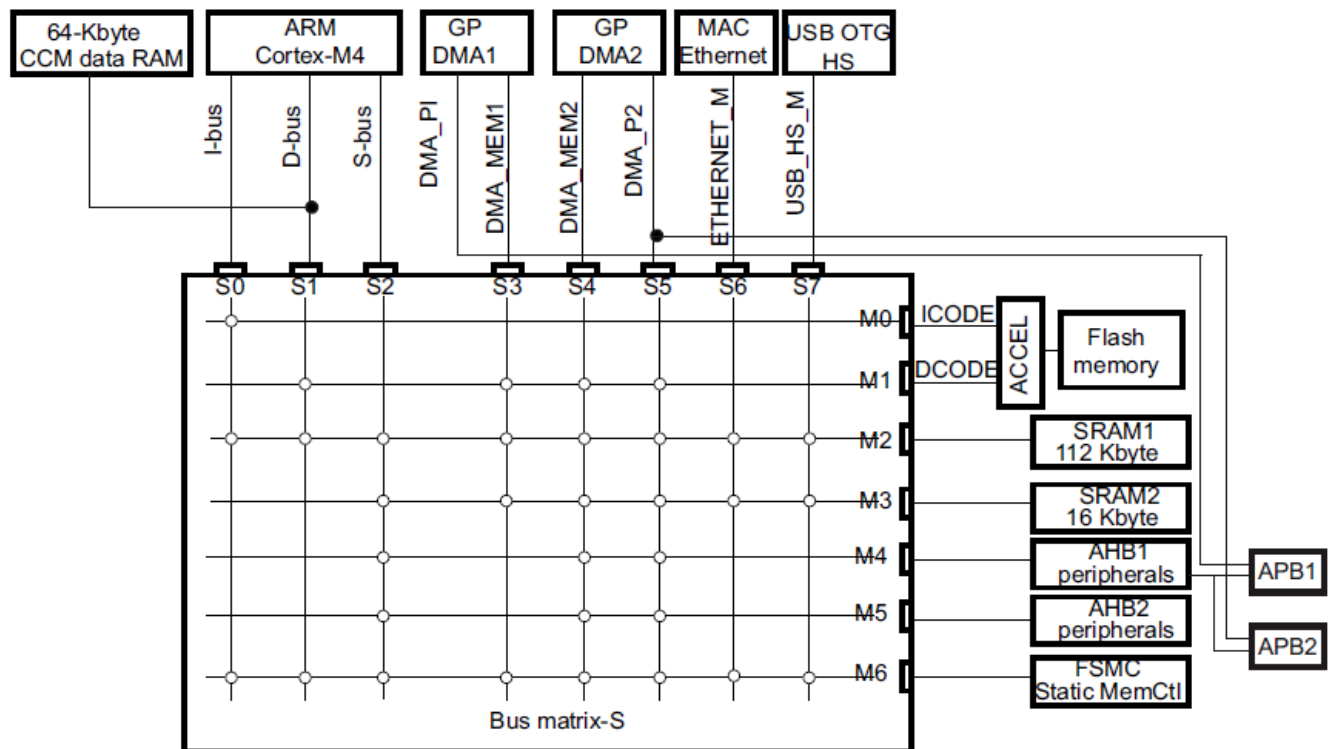


Рис.37. Мульти-АВБ матриця шин 32 р

КПДП (DMA)
memory-to-memory, peripheral-to-memory and
memory-to-peripheral

- SPI and I2S
- I2C
- USART
- General-purpose, basic and advanced-control timers TIMx
- DAC
- SDIO
- Camera interface (DCMI)
- ADC.

Живлення

- VDD = 1.8 to 3.6 V: external power supply for I/Os and the internal regulator (when enabled), provided externally through VDD pins.
- VSSA, VDDA = 1.8 to 3.6 V: external analog power supplies for ADC, DAC, Reset blocks, RCs and PLL. VDDA and VSSA must be connected to VDD and VSS, respectively.
- VBAT = 1.65 to 3.6 V: power supply for RTC, external clock 32 kHz oscillator and backup registers (through power switch) when VDD is not present.

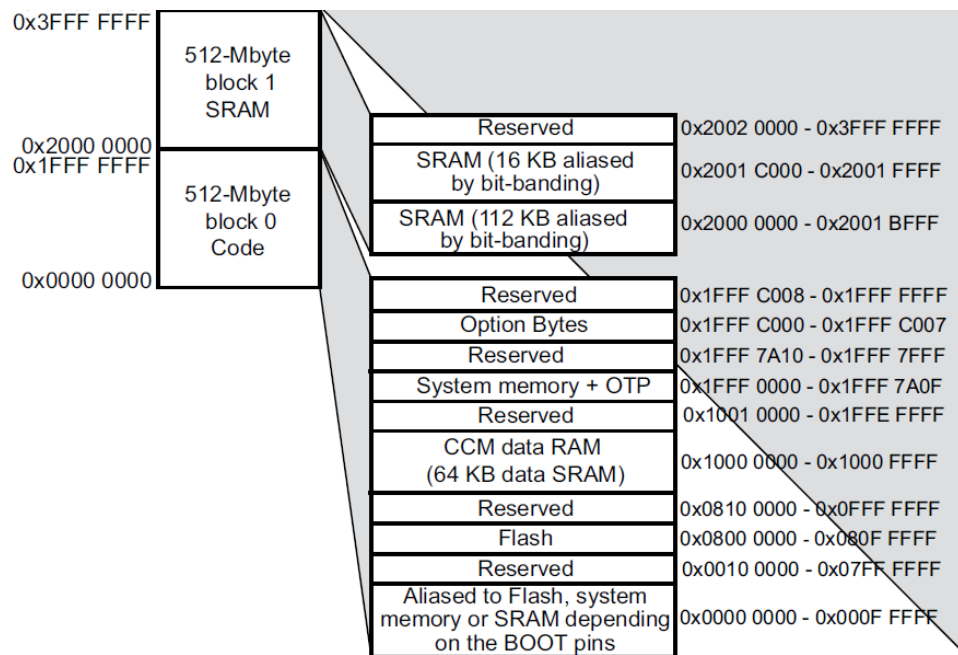


Рис.38а. Структура пам'яті STM32F40x

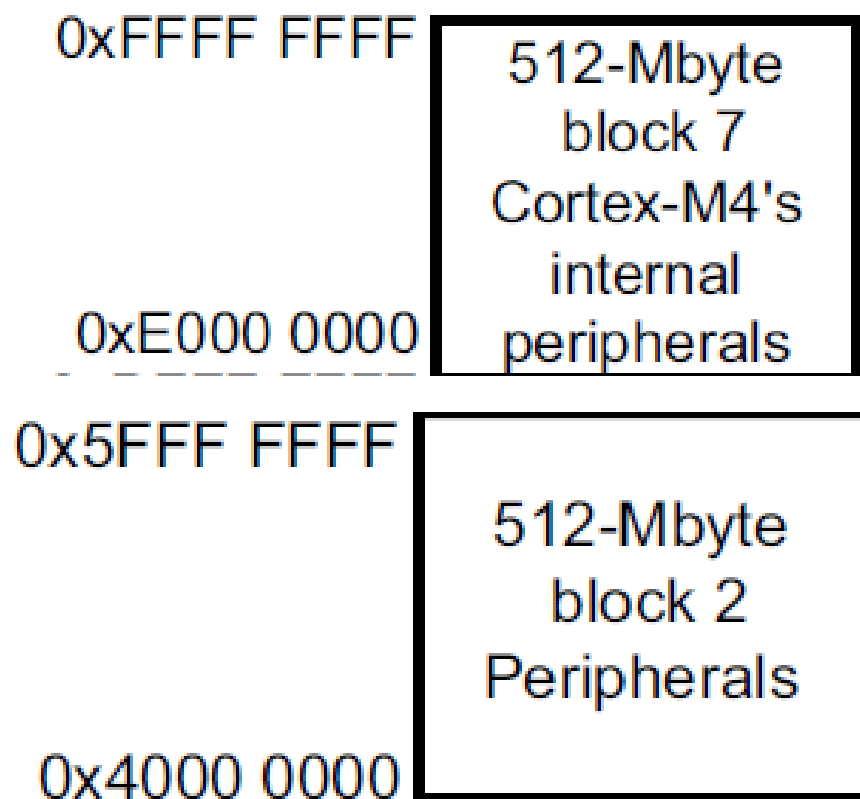


Рис.38b. Структура пам'яті STM32F40x

Bus	Boundary address	Peripheral
	0xE00F FFFF - 0xFFFF FFFF	Reserved
Cortex-M4	0xE000 0000 - 0xE00F FFFF	Cortex-M4 internal peripherals
	0xA000 1000 - 0xDFFF FFFF	Reserved
AHB3	0xA000 0000 - 0xA000 0FFF	FSMC control register
	0x9000 0000 - 0x9FFF FFFF	FSMC bank 4
	0x8000 0000 - 0x8FFF FFFF	FSMC bank 3
	0x7000 0000 - 0x7FFF FFFF	FSMC bank 2
	0x6000 0000 - 0x6FFF FFFF	FSMC bank 1
	0x5006 0C00- 0x5FFF FFFF	Reserved
AHB2	0x5006 0800 - 0x5006 0BFF	RNG
	0x5005 0400 - 0x5006 07FF	Reserved
	0x5005 0000 - 0x5005 03FF	DCMI
	0x5004 0000- 0x5004 FFFF	Reserved
	0x5000 0000 - 0x5003 FFFF	USB OTG FS
	0x4008 0000- 0x4FFF FFFF	Reserved

Рис.39a. Регістри STM32F40x

Bus	Boundary address	Peripheral
AHB1	0x4004 0000 - 0x4007 FFFF	USB OTG HS
	0x4002 9400 - 0x4003 FFFF	Reserved
	0x4002 9000 - 0x4002 93FF	ETHERNET MAC
	0x4002 8C00 - 0x4002 8FFF	
	0x4002 8800 - 0x4002 8BFF	
	0x4002 8400 - 0x4002 87FF	
	0x4002 8000 - 0x4002 83FF	
	0x4002 6800 - 0x4002 7FFF	Reserved
	0x4002 6400 - 0x4002 67FF	DMA2
	0x4002 6000 - 0x4002 63FF	DMA1
	0x4002 5000 - 0x4002 5FFF	Reserved
	0x4002 4000 - 0x4002 4FFF	BKPSRAM
	0x4002 3C00 - 0x4002 3FFF	Flash interface register
	0x4002 3800 - 0x4002 3BFF	RCC
	0x4002 3400 - 0x4002 37FF	Reserved
	0x4002 3000 - 0x4002 33FF	CRC
	0x4002 2400 - 0x4002 2FFF	Reserved
	0x4002 2000 - 0x4002 23FF	GPIOI
	0x4002 1C00 - 0x4002 1FFF	GPIOH
	0x4002 1800 - 0x4002 1BFF	GPIOG
	0x4002 1400 - 0x4002 17FF	GPIOF
	0x4002 1000 - 0x4002 13FF	GPIOE
	0x4002 0C00 - 0x4002 0FFF	GPIOD
	0x4002 0800 - 0x4002 0BFF	GPIOC
	0x4002 0400 - 0x4002 07FF	GPIOB
	0x4002 0000 - 0x4002 03FF	GPIOA
	0x4001 5800 - 0x4001 FFFF	Reserved

Рис.39b. Регістри STM32F40x

Регістри паралельних портів

GPIO Registers

Configuration Registers

STM32 містить чотири регістри конфігурації для кожного з портів:

- Port mode register – GPIOx_MODER
- Output type register – GPIOx_OTYPER
- Speed register – GPIOx_OSPEEDR
- Pull-up/Pull-down register – GPIOx_PUPDR

Кожен з цих регістрів має довжину 32 біти, хоча і не всі біти використовуються у всіх регістрах.

Port Mode Register – GPIOx_MODER

Цей 32-розрядний регістр має 2 бітні значення, які визначають режим роботи.

Можна встановити такі режими:

Bit Values	Description
00	Input
01	General purpose output
10	Alternate function
11	Analog

Port	Reset Value
A	0xA800 0000
B	0x0000 0280
All others	0x0000 0000

Output Type Register – GPIOx_OTYPER

Bit Value	Description
0	Push/pull output
1	Open drain output

Speed Register – GPIOx_OSPEEDR

Bit Values	Description
00	2 MHz Low speed
01	25 MHz Medium speed
10	50 MHz Fast speed
11	100 MHz High speed on 30pF 80 MHz High speed on 15 pF

Pull-up/Pull-down register – GPIOx_PUPDR

Bit Values	Description
00	No pull-up or pull-down resistor
01	Pull-up resistor
10	Pull-down resistor
11	Reserved

Input data register – GPIOx_IDR

Bit Value	Description
0	High logic value
1	Low logic value

Output data register – GPIOx_ODR

Bit Value	Description
0	High logic value
1	Low logic value

Вузол світлодіодів модуля STM32F4DISCOVERY

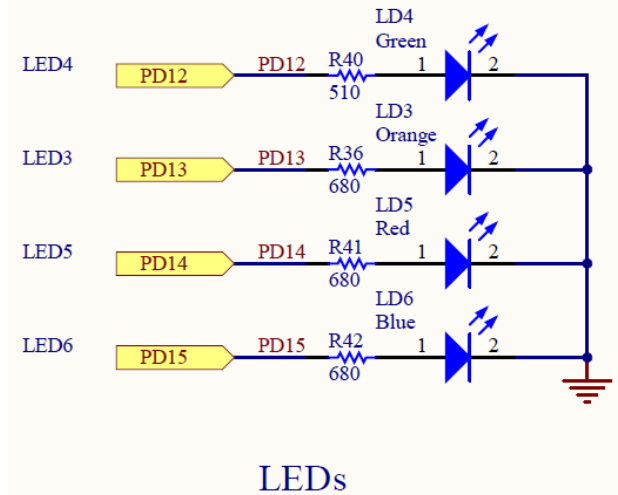


Рис.40. Схема вузла світлодіодів модуля STM32F4DISCOVERY

Процедура ініціалізації порта PD до якого підключені світлодіоди

```
void UB_Led_Init(void)
{
    GPIO_InitTypeDef  GPIO_InitStructure;
    LED_NAME_t led_name;

    for(led_name=0;led_name<LED_ANZ;led_name++) {
        // Clock Enable
        RCC_AHB1PeriphClockCmd(LED[led_name].LED_CLK, ENABLE);

        // Config
        GPIO_InitStructure.GPIO_Pin = LED[led_name].LED_PIN;
        GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
        GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
        GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
        GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
        GPIO_Init(LED[led_name].LED_PORT, &GPIO_InitStructure);

        // Default
        if(LED[led_name].LED_INIT==LED_OFF) {
            UB_Led_Off(led_name);
        }
        else {
            UB_Led_On(led_name);
        }
    }
}
```

Включення-виключення світлодіодів

```
void UB_Led_Off(LED_NAME_t led_name)
{
    LED[led_name].LED_PORT->BSRRH = LED[led_name].LED_PIN;
}

void UB_Led_On(LED_NAME_t led_name)
{
    LED[led_name].LED_PORT->BSRRL = LED[led_name].LED_PIN;
}
```

Вузол реле лабораторного стенда

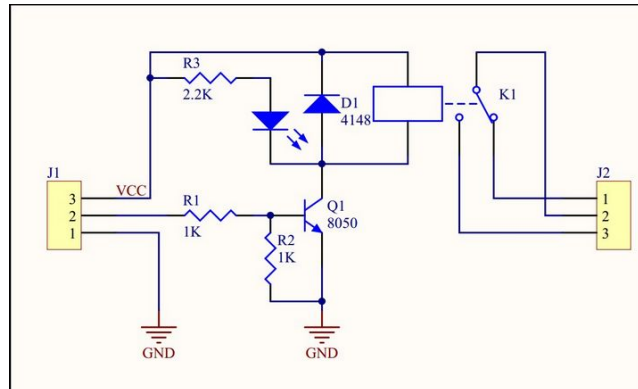
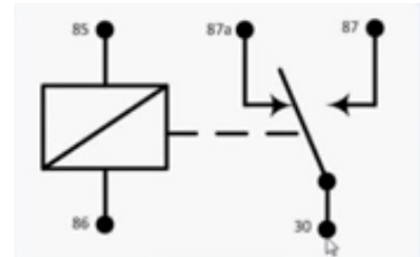


Рис41. Вузол реле лабораторного стенду

Лінія керування реле J1/2 підключається до лінії 2 порта E

Ініціалізація лінії порта E.2

```
/* Configure PE2 in output pushpull mode */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(GPIOE, &GPIO_InitStructure);
```

Керування реле

```
GPIO_WriteBit(GPIOE,GPIO_Pin_2,Bit_RESET); //RL On
GPIO_WriteBit(GPIOE,GPIO_Pin_2,Bit_SET);   //RL Off
```

Вузол RS-485

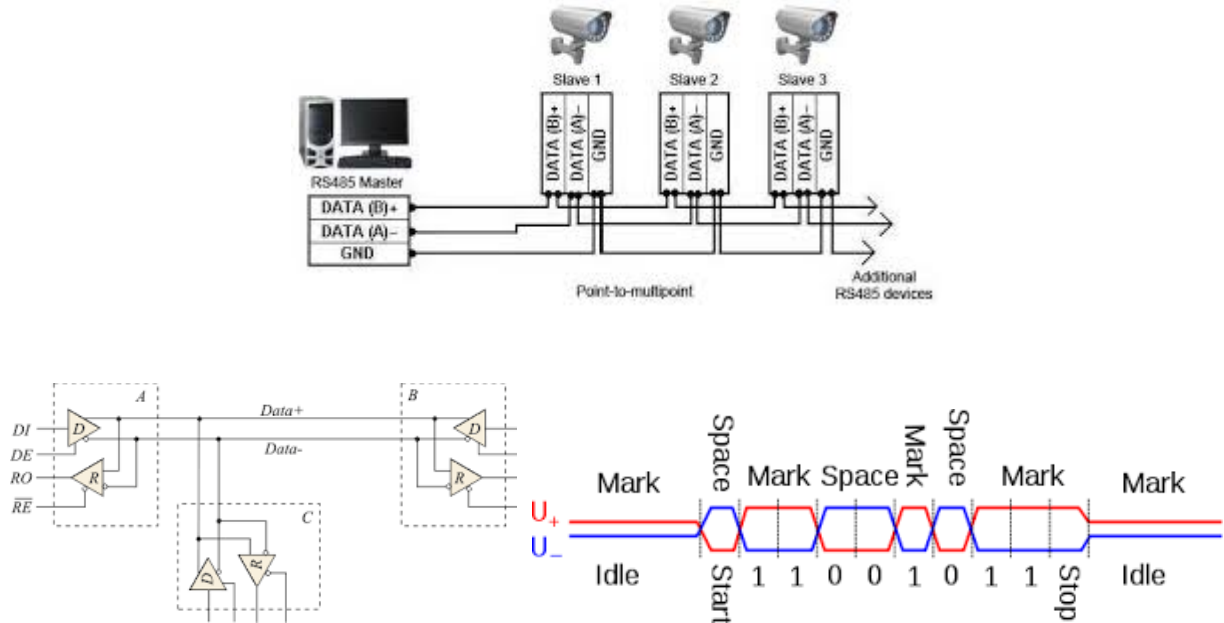


Рис.42. Інтерфейс RS-485

Обмін інформацією між модулем STM32F4DISCOVERY та ПК здійснюється через порт UART2 мікроконтролера STM32F407 модуля STM32F4DISCOVERY та адаптер USB-RS485 підключений до ПК.



Рис.43. Зовнішній вигляд вузла RS-485 лабораторного стенду

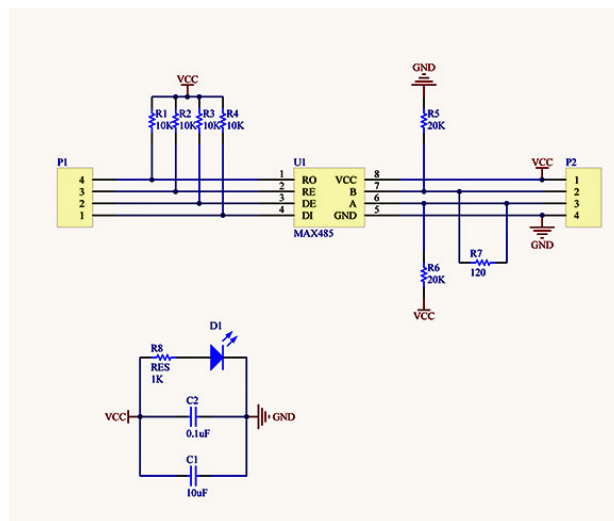


Рис.44. Схема вузла RS-485 лабораторного стенду

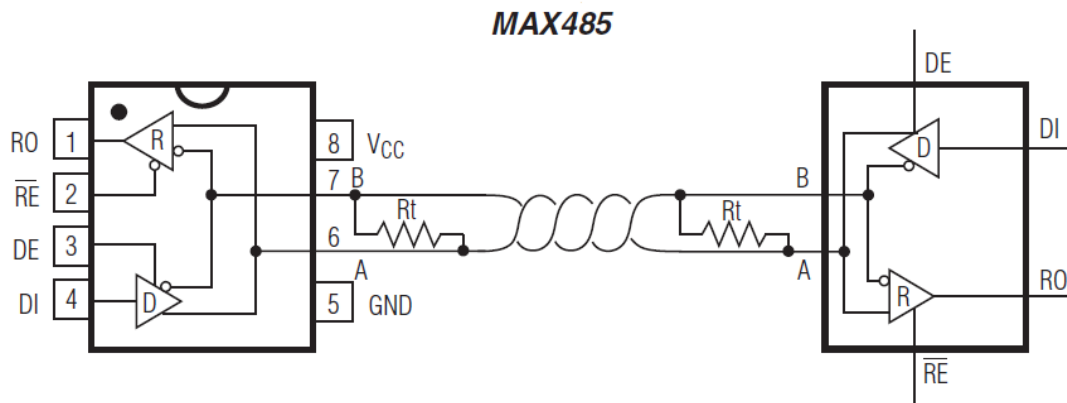


Рис.45. Мікросхема MAX485 перетворювача UART-RS485



Рис.46. Зовнішній вигляд адаптера USB-RS485

Входи RE/DE м/сх MAX485 керуються лінією 1 порта A.

Ініціалізація лінії керування:

```
/* Configure PA1 in output pushpull mode */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

Включення режиму «передача» модуль → ПК:

```
GPIO_WriteBit(GPIOA,GPIO_Pin_1,Bit_SET); //RS-485
```

Включення режиму «читання» модуль ← ПК:

```
GPIO_WriteBit(GPIOA,GPIO_Pin_1,Bit_SET); //RS-485
```

Ініціалізація порта UART2:

```
void UB_Uart_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;
    NVIC_InitTypeDef NVIC_InitStructure;
    UART_NAME_t nr;

    for(nr=0;nr<UART_ANZ;nr++) {

        // Clock enable TX RX Pins
        RCC_AHB1PeriphClockCmd(UART[nr].TX.CLK, ENABLE);
        RCC_AHB1PeriphClockCmd(UART[nr].RX.CLK, ENABLE);
```



```

// Clock enable UART
if((UART[nr].UART==USART1) || (UART[nr].UART==USART6)) {
    RCC_APB2PeriphClockCmd(UART[nr].CLK, ENABLE);
}
else {
    RCC_APB1PeriphClockCmd(UART[nr].CLK, ENABLE);
}

// UART Alternative-Funktions IO-Pins
GPIO_PinAFConfig(UART[nr].TX.PORT,UART[nr].TX.SOURCE,UART[nr].AF);
GPIO_PinAFConfig(UART[nr].RX.PORT,UART[nr].RX.SOURCE,UART[nr].AF);

// UART Alternative-Funktion PushPull
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
// TX-Pin
GPIO_InitStructure.GPIO_Pin = UART[nr].TX.PIN;
GPIO_Init(UART[nr].TX.PORT, &GPIO_InitStructure);
// RX-Pin
GPIO_InitStructure.GPIO_Pin = UART[nr].RX.PIN;
GPIO_Init(UART[nr].RX.PORT, &GPIO_InitStructure);

USART_OverSampling8Cmd(UART[nr].UART, ENABLE);

// init Baudrate, 8Databits, 1Stopbit, No Parity, No RTS+CTS
USART_InitStructure.USART_BaudRate = UART[nr].BAUD;
USART_InitStructure.USART_WordLength = USART_WordLength_8b;
USART_InitStructure.USART_StopBits = USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
USART_Init(UART[nr].UART, &USART_InitStructure);

// UART enable
USART_Cmd(UART[nr].UART, ENABLE);

// RX-Interrupt enable
USART_ITConfig(UART[nr].UART, USART_IT_RXNE, ENABLE);

// enable UART Interrupt-Vector
NVIC_InitStructure.NVIC_IRQChannel = UART[nr].INT;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);

// RX-Puffer
UART_RX[nr].rx_buffer[0]=RX_END_CHR;
UART_RX[nr].wr_ptr=0;
UART_RX[nr].rd_ptr=0;
UART_RX[nr].status=RX_EMPTY;
}
}

```

Передача інформації з буфера «buf» через UART в ПК:
 UB_Uart_SendString(COM2,buf,LFCR);

FM24CL16

16K bit Ferroelectric Nonvolatile RAM

- Organized as 2,048 x 8 bits
- Unlimited Read/Write Cycles
- 10 year Data Retention
- NoDelay™ Writes
- Advanced High-Reliability Ferroelectric Process

Fast Two-wire Serial Interface

- Up to 1MHz maximum bus frequency
- Direct hardware replacement for EEPROM

Low Power Operation

- **New** 2.7 - 3.6V operation
- 75 μ A Active Current (100 kHz) @ 3V
- 1 μ A Standby Current

Industry Standard Configuration

- Industrial Temperature -40° C to +85° C
- 8-pin SOIC

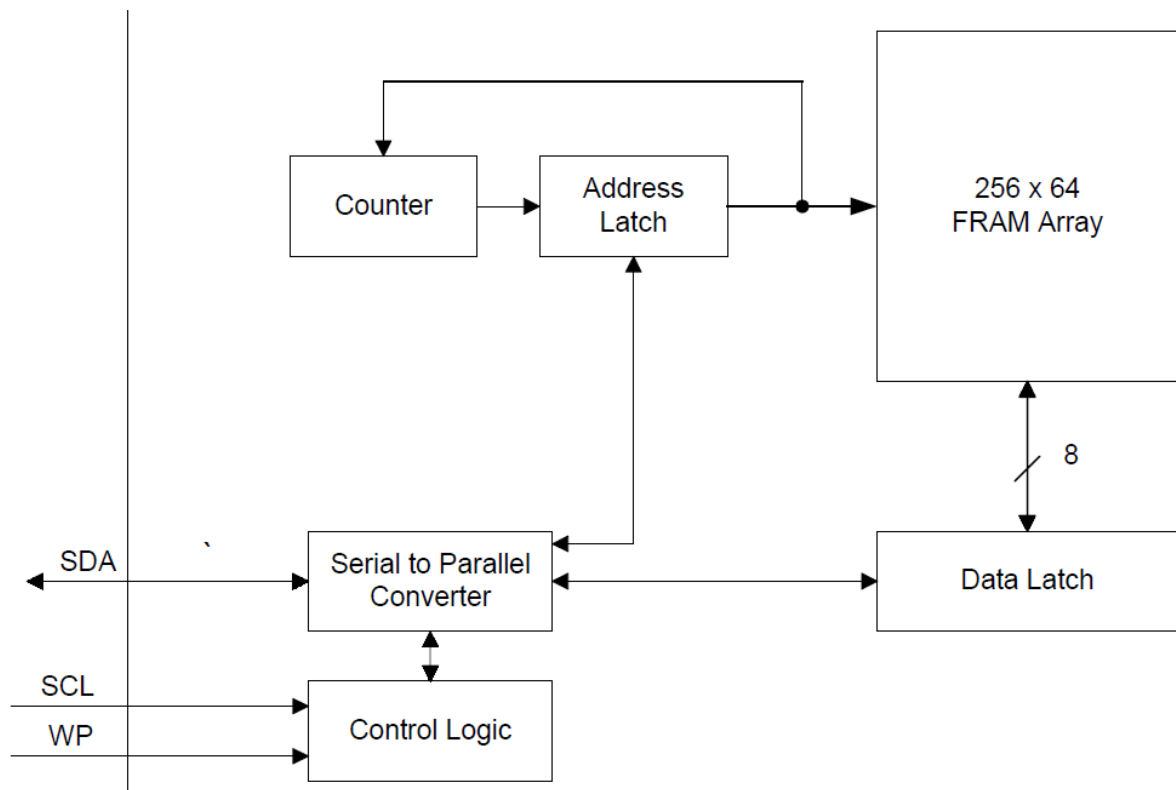
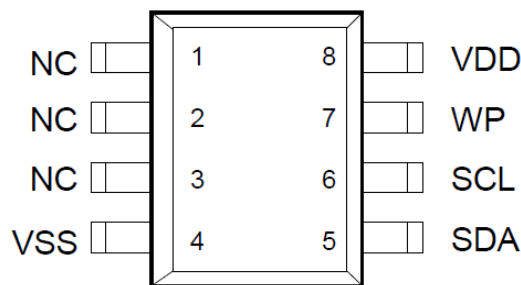


Рис.47. Внутрішня структура FM24CL16



Pin Names	Function
SDA	Serial Data/Address
SCL	Serial Clock
WP	Write Protect
VDD	Supply Voltage
VSS	Ground

Pin Name	Type	Pin Description
SDA	I/O	Serial Data Address: This is a bi-directional data pin for the two-wire interface. It employs an open-drain output and is intended to be wire-OR'd with other devices on the two-wire bus. The input buffer incorporates a Schmitt trigger for noise immunity and the output driver includes slope control for falling edges. A pull-up resistor is required.
SCL	Input	Serial Clock: The serial clock input for the two-wire interface. Data is clocked-out on the falling edge and clocked-in on the rising edge.
WP	Input	Write Protect: When WP is high, the entire array is write-protected. When WP is low, all addresses may be written. This pin is internally pulled down.
VDD	Supply	Supply Voltage (3V)
VSS	Supply	Ground
NC	-	No connect

Рис.48. Контакти FM24CL16

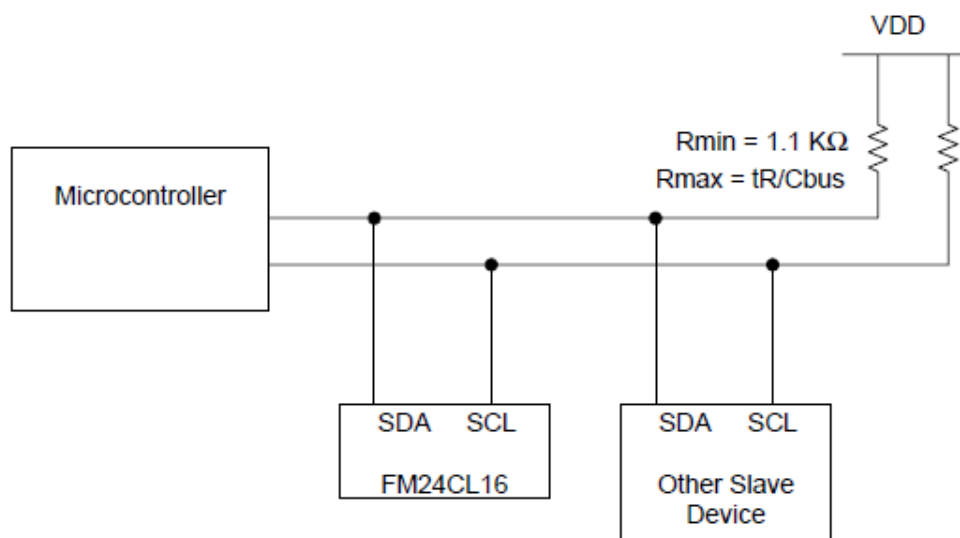


Рис.49. Підключення FM24CL16 до мікроконтролера

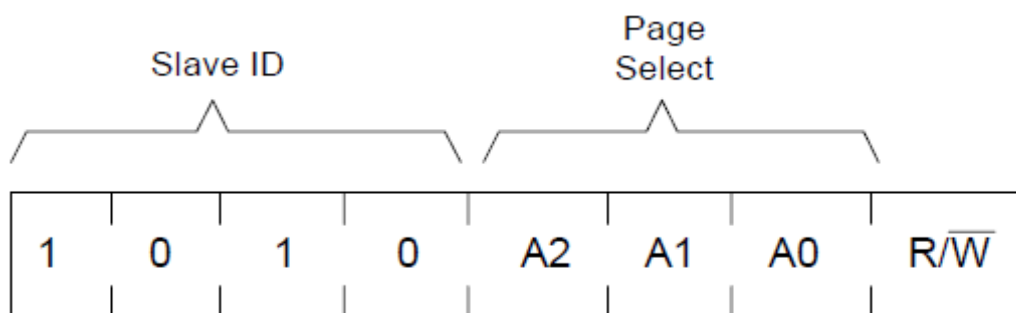


Рис.50. Адресація FM24CL16

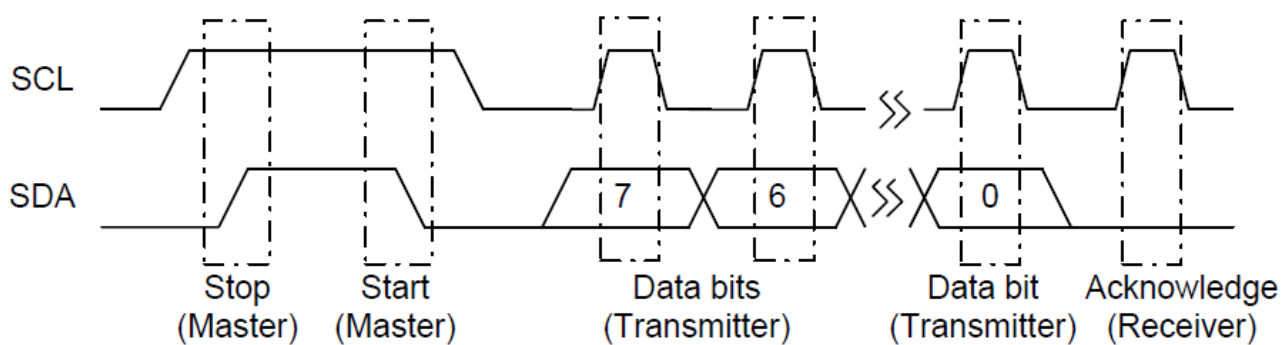


Рис.51. Діаграма запису даних в FM24CL16

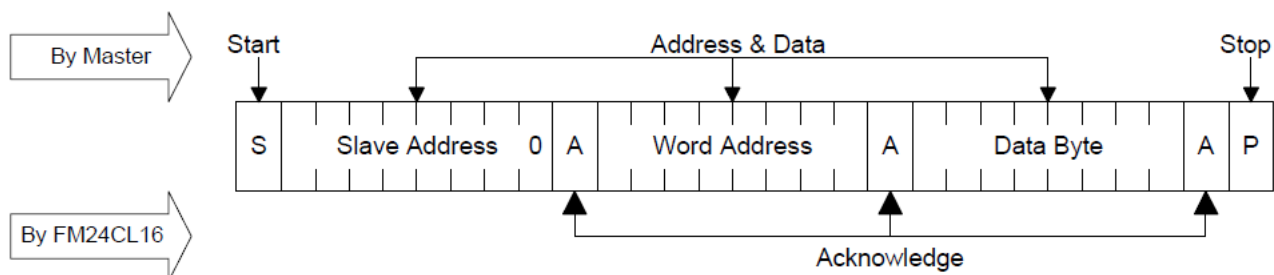


Рис.52. Запис одиночного байта в FM24CL16

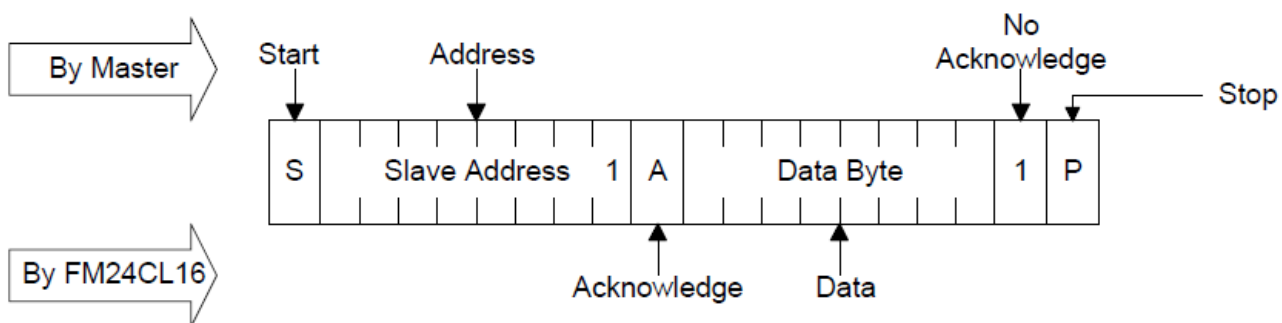


Рис.53. Читання одиночного байта з FM24CL16

Інтерфейс I2C STM32F407

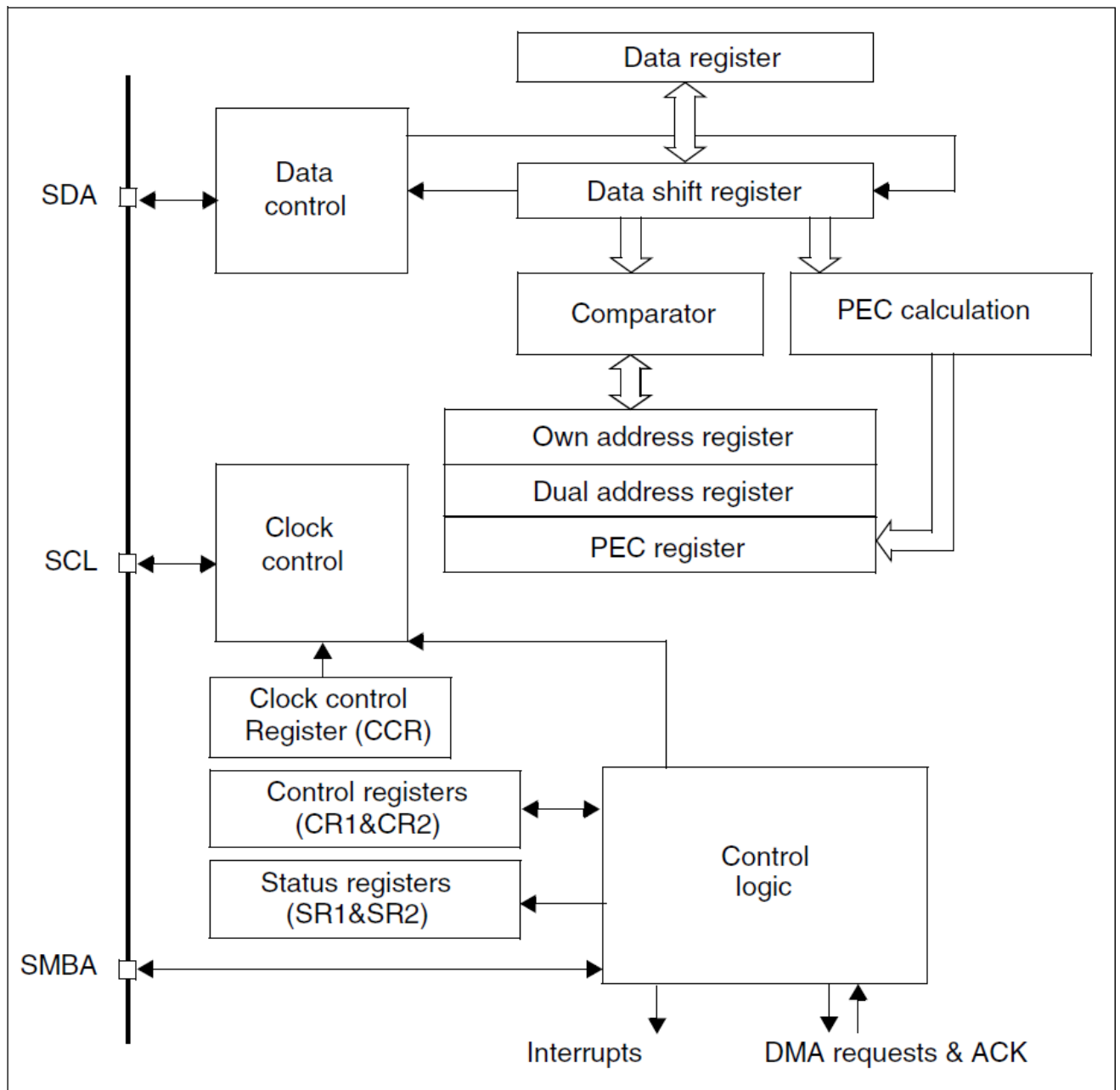
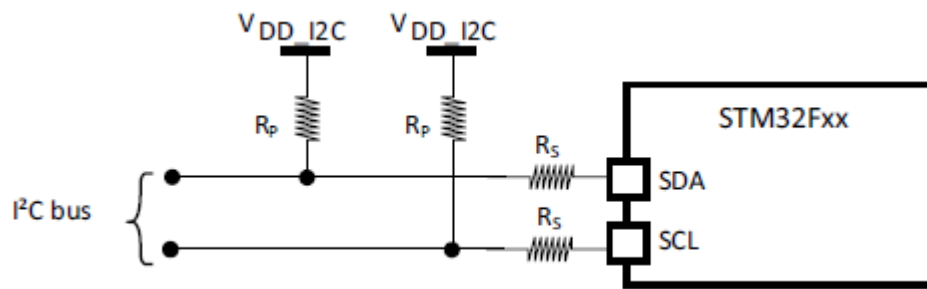
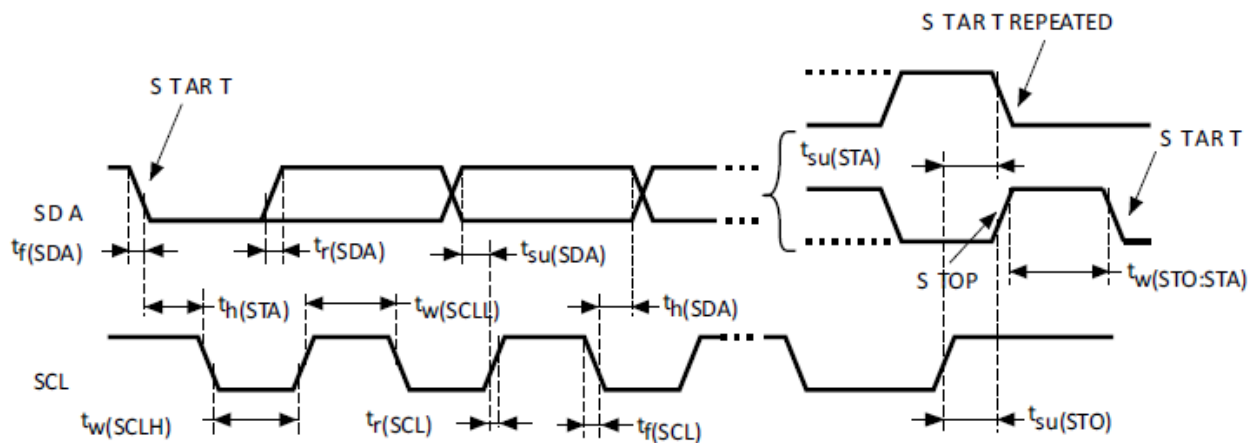


Рис.54. Внутрішня структура вузла I2C STM32F407



PB6	I/O	FT	I2C1_SCL/ TIM4_CH1 / CAN2_TX / DCMI_D5/USART1_TX/ EVENTOUT
PB7	I/O	FT	I2C1_SDA / FSMC_NL / DCMI_VSYNC / USART1_RX/ TIM4_CH2/ EVENTOUT

Рис.55. Лінії I2C1 STM32F407



Symbol	Parameter	Standard mode I ² C ⁽¹⁾		Fast mode I ² C ⁽¹⁾⁽²⁾		Unit
		Min	Max	Min	Max	
$t_h(STA)$	Start condition hold time	4.0	-	0.6	-	μs
$t_{su}(STA)$	Repeated Start condition setup time	4.7	-	0.6	-	
$t_{su}(STO)$	Stop condition setup time	4.0	-	0.6	-	μs
$t_w(STO:STA)$	Stop to Start condition time (bus free)	4.7	-	1.3	-	μs
C_b	Capacitive load for each bus line	-	400	-	400	pF

Рис.56. Часова діаграма вузла I2C STM32F407

Регістри I2C STM32F407

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x00	I2C_CR1	Reserved																SWRST	Reserved	ALERT	PEC	POS	ACK	STOP	START	NOSTRETCH	ENGCG	ENPEC	ENARP	SMBTYPE	Reserved	SMBUS	PE			
	Reset value																	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x04	I2C_CR2	Reserved																LAST			DMAEN	ITBUFEN	ITEVTEN	ITERREN	Reserved	FREQ[5:0]										
	Reset value																	0			0	0	0	0	0		0	0	0	0	0	0	0			
0x08	I2C_OAR1	Reserved																ADDMODE	Reserved				ADD[9:8]		ADD[7:1]					ADD0						
	Reset value																	0					0	0	0					0						
0x0C	I2C_OAR2	Reserved																ADD2[7:1]					ENDUAL													
	Reset value																	0					0	0	0	0	0	0	0	0						
0x10	I2C_DR	Reserved																DR[7:0]																		
	Reset value																	0					0	0	0	0	0	0	0	0						
0x14	I2C_SR1	Reserved																SMBALERT	TIMEOUT	Reserved	PECERR	OVR	AF	ARLO	BERR	TxE	RxNE	Reserved	STOPF	ADD10	BTF	ADDR	SB			
	Reset value																	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0			
0x18	I2C_SR2	Reserved																PEC[7:0]					DUALF	SMBHOST	SMBDEFAULT	GENCALL	Reserved	TRA	BUSY	MSL						
	Reset value																	0					0	0	0	0		0	0	0						
0x1C	I2C_CCR	Reserved																F/S	DUTY	Reserved	CCR[11:0]															
	Reset value																	0	0		0											0	0	0	0	0
0x20	I2C_TRISE	Reserved																TRIASE[5:0]																		
	Reset value																	0					0	0	0	0	1	0								
0x24	I2C_FLTR	Reserved																ANOFF					DNF[3:0]													
	Reset value																	0					0	0	0	0	0									

Процедура ініціалізації I2C1

```
// SCL PB6
// SDA PB7

void UB_I2C1_Init(void)
{
    static uint8_t init_ok=0;
    GPIO_InitTypeDef GPIO_InitStructure;

    // I2C-Clock enable
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_I2C1, ENABLE);

    // Clock Enable Pins
    RCC_AHB1PeriphClockCmd(I2C1DEV.SCL.CLK, ENABLE);
    RCC_AHB1PeriphClockCmd(I2C1DEV.SDA.CLK, ENABLE);

    // I2C reset
    RCC_APB1PeriphResetCmd(RCC_APB1Periph_I2C1, ENABLE);
    RCC_APB1PeriphResetCmd(RCC_APB1Periph_I2C1, DISABLE);

    // I2C Alternative-Funktions IO-Pins
    GPIO_PinAFConfig(I2C1DEV.SCL.PORT, I2C1DEV.SCL.SOURCE, GPIO_AF_I2C1);
    GPIO_PinAFConfig(I2C1DEV.SDA.PORT, I2C1DEV.SDA.SOURCE, GPIO_AF_I2C1);

    // I2C Alternative-Funktion
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;

    // SCL-Pin
    GPIO_InitStructure.GPIO_Pin = I2C1DEV.SCL.PIN;
    GPIO_Init(I2C1DEV.SCL.PORT, &GPIO_InitStructure);
    // SDA-Pin
    GPIO_InitStructure.GPIO_Pin = I2C1DEV.SDA.PIN;
    GPIO_Init(I2C1DEV.SDA.PORT, &GPIO_InitStructure);

    // I2C init
    P_I2C1_InitI2C();
}
```

Процедура запису байта

```
int16_t UB_I2C1_WriteByte(uint8_t slave_adr, uint8_t adr, uint8_t wert)
{
    int16_t ret_wert=0;
    uint32_t timeout=I2C1_TIMEOUT;

    // Start
    I2C_GenerateSTART(I2C1, ENABLE);

    timeout=I2C1_TIMEOUT;
    while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_SB)) {
        if(timeout!=0) timeout--; else return(P_I2C1_timeout(-1));
    }

    // Slave-Adresse (write)
    I2C_Send7bitAddress(I2C1, slave_adr, I2C_Direction_Transmitter);

    timeout=I2C1_TIMEOUT;
    while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_ADDR)) {
```



```

    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-2));
}

// ADDR-Flag
I2C1->SR2;

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_TXE)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-3));
}

// Adresse write
I2C_SendData(I2C1, adr);

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_TXE)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-4));
}

// Daten write
I2C_SendData(I2C1, wert);

timeout=I2C1_TIMEOUT;
while ((!I2C_GetFlagStatus(I2C1, I2C_FLAG_TXE)) || (!I2C_GetFlagStatus(I2C1,
I2C_FLAG_BTF))) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-5));
}

// Stop-
I2C_GenerateSTOP(I2C1, ENABLE);

ret_wert=0; // ok

return(ret_wert);
}

```

Процедура читання байта

```

int16_t UB_I2C1_ReadByte(uint8_t slave_adr, uint8_t adr)
{
    int16_t ret_wert=0;
    uint32_t timeout=I2C1_TIMEOUT;

    // Start-
    I2C_GenerateSTART(I2C1, ENABLE);

    timeout=I2C1_TIMEOUT;
    while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_SB)) {
        if(timeout!=0) timeout--; else return(P_I2C1_timeout(-1));
    }

    // ACK disable
    I2C_AcknowledgeConfig(I2C1, DISABLE);

    // Slave-Adresse (write)
    I2C_Send7bitAddress(I2C1, slave_adr, I2C_Direction_Transmitter);

    timeout=I2C1_TIMEOUT;
    while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_ADDR)) {
        if(timeout!=0) timeout--; else return(P_I2C1_timeout(-2));
    }
}

```

```

// ADDR-Flag
I2C1->SR2;

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_TXE)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-3));
}

// Adresse write
I2C_SendData(I2C1, adr);

timeout=I2C1_TIMEOUT;
while ((!I2C_GetFlagStatus(I2C1, I2C_FLAG_TXE)) || (!I2C_GetFlagStatus(I2C1,
I2C_FLAG_BTF))) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-4));
}

// Start- write
I2C_GenerateSTART(I2C1, ENABLE);

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_SB)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-5));
}

// Slave-Adresse write (read)
I2C_Send7bitAddress(I2C1, slave_adr, I2C_Direction_Receiver);

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_ADDR)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-6));
}

// ADDR-Flag
I2C1->SR2;

timeout=I2C1_TIMEOUT;
while (!I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)) {
    if(timeout!=0) timeout--; else return(P_I2C1_timeout(-7));
}

// Stop-
I2C_GenerateSTOP(I2C1, ENABLE);

// Daten
ret_wert=(int16_t)(I2C_ReceiveData(I2C1));

// ACK enable
I2C_AcknowledgeConfig(I2C1, ENABLE);

return(ret_wert);
}

```

Література

1. Reference manual. STM32F405xx/07xx, STM32F415xx/17xx, STM32F42xxx and
2. STM32F43xxx advanced ARM®-based 32-bit MCUs.pdf
3. https://www.st.com/content/st_com/en/search.html?q=Reference%20manual.%20STM32F405xx/07xx-t=resources-page=1
4. STM32F4DISCOVERY
5. https://www.st.com/content/st_com/en/search.html?q=STM32F4DISCOVERY%20-t=resources-page=1
6. FM24CL16-G-ETC.pdf
7. MAX485.pdf
8. Конспект лекцій з дисципліни «Мікропроцесорні системи»

НАВЧАЛЬНЕ ВИДАННЯ

**Дослідження режимів керування компонентами МПС
з допомогою паралельного та послідовного інтерфейсів**

Методичні вказівки
до лабораторної роботи № 5 з курсу “ Мікропроцесорні системи ”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Укладач

Пуйда В. Я., к. т. н, доцент

Редактор

Комп’ютерне верстання

Здано у видавництво . Підписано до друку
Формат 70х100/16. Папір офсетний. Друк на різнографі
Умовн. друк. арк. Обл.-вид. арк..
Тираж прим. Зам..

Видавництво Національного університету “Львівська політехніка”
Регістраційне свідоцтво ДК №751 від 27.12.2001 р.

Поліграфічний центр Видавництва
Національного університету “Львівська політехніка”

Вул.. Ф. Колесси, 2. Львів, 79000